

ANALISIS OPTIMASI REALTIME PIPELINE NVIDIA DEEPSTREAM UNTUK APLIKASI ADAS BERBASIS EDGE DEVICE



**PERDI
D121221015**

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN
MAKASSAR
2026**



**ANALISIS OPTIMASI REALTIME PIPELINE NVIDIA DEEPSTREAM UNTUK
APLIKASI ADAS BERBASIS EDGE DEVICE**

**PERDI
D121221015**



**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN
2026**

**ANALISIS OPTIMASI REALTIME PIPELINE NVIDIA DEEPSTREAM UNTUK
APLIKASI ADAS BERBASIS EDGE DEVICE**

PERDI
D121221015

Skripsi

sebagai salah satu syarat untuk mencapai gelar sarjana

Program Studi Teknik Informatika

pada

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS HASANUDDIN
MAKASSAR
2026**

**ANALISIS OPTIMASI REALTIME PIPELINE NVIDIA DEEPSTREAM UNTUK
APLIKASI ADAS BERBASIS EDGE DEVICE**

PERDI
D121221015

Skripsi,

telah dipertahankan di depan Panitia Ujian Sarjana XXXXX pada tanggal bulan tahun
dan dinyatakan telah memenuhi syarat kelulusan pada

Program Studi Teknik Informatika
Departemen Teknik Informatika
Fakultas Teknik
Universitas Hasanuddin
Makassar

Mengesahkan:

Pembimbing tugas akhir,

Mengetahui::

Ketua Program Studi,,

Dr. Muhammad Abdillah Rahmat,
S.T., M.T.
NIP 199510262024123001

Prof. Dr. Ir. Indrabayu, ST., MT.,
M.Bus.Sys., IPM, ASEAN, Eng.
NIP 197507162002121004

PERNYATAAN KEASLIAN SKRIPSI DAN PELIMPAHAN HAK CIPTA

Dengan ini saya menyatakan bahwa, skripsi berjudul “**Analisis Optimasi Realtime Pipeline Nvidia Deepstream Untuk Aplikasi Adas Berbasis Edge Device**” adalah benar karya saya dengan arahan dari pembimbing Dr. Muhammad Abdillah Rahmat, S.T., M.T. sebagai Pembimbing Utama. Karya ilmiah ini belum diajukan dan tidak sedang diajukan dalam bentuk apa pun kepada perguruan tinggi mana pun. Sumber informasi yang berasal atau dikutip dari karya yang diterbitkan maupun tidak diterbitkan dari penulis lain telah disebutkan dalam teks dan dicantumkan dalam Daftar Pustaka skripsi ini. Apabila di kemudian hari terbukti atau dapat dibuktikan bahwa sebagian atau keseluruhan skripsi ini adalah karya orang lain, maka saya bersedia menerima sanksi atas perbuatan tersebut berdasarkan aturan yang berlaku.

Dengan ini saya melimpahkan hak cipta (hak ekonomis) dari karya tulis saya berupa skripsi ini kepada Universitas Hasanuddin.

Makassar, 20 Agustus 2026

PERDI
NIM D121221015

UCAPAN TERIMA KASIH

Puji syukur penulis panjatkan kehadiran Allah SWT atas segala limpahan rahmat, taufik, dan hidayah-Nya sehingga penulis dapat menyelesaikan skripsi yang berjudul "Sistem Deteksi Sampah di Permukaan Sungai Menggunakan Metode YOLO11 dan YOLOv8 Terintegrasi GPS" sebagai salah satu syarat untuk mencapai gelar Sarjana pada Program Studi Teknik Informatika, Fakultas Teknik, Universitas Hasanuddin. Penulis menyadari bahwa proses penyusunan skripsi ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, dengan segala kerendahan hati penulis menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Prof. Dr. Ir. Indrabayu, S.T., M.T., M.Bus.Sys., IPM., ASEAN. Eng selaku pembimbing tugas akhir yang telah memberikan arahan, masukan, dan motivasi yang sangat berarti selama proses penelitian dan penulisan skripsi ini.
2. Ibu Elly Warni, ST., MT. dan Bapak Dr. Muhammad Abdillah Rahmat, S.T., M.T. selaku tim penguji tugas akhir, atas masukan, evaluasi, serta saran yang konstruktif sehingga tugas akhir ini dapat disusun dengan lebih baik.
3. Segenap Dosen Program Studi Teknik Informatika Universitas Hasanuddin yang telah mendidik dan memberikan ilmu pengetahuan selama masa perkuliahan serta Tenaga Kependidikan yang telah memberikan pelayanan akademik yang maksimal selama penulis menempuh studi.
4. Kedua orang tua penulis, Prof. Dr. rer. nat. Wira Bahari Nurdin dan Prof. Dr. rer. nat. Marianti Manggau, Apt. yang telah memberikan doa, dukungan moral, dan menjadi panutan penulis menjalani studi, serta kakak penulis Kak Jungman yang menjadi inspirasi penulis untuk menempuh bidang keinformatikaan ini, serta adik penulis Amirul yang menjadi saudara seperjuangan.
5. Seluruh *core team* GDGoC Unhas yang telah menjadi *core memory* penulis, terutama Sri, Eka, Nisa, dan Jo yang telah menjadi sahabat penulis di perkuliahan.
6. Teman-teman kelas A terutama Fathin, Wine, Ennu, Lala, Zaky, dan Mirsa, serta saudara AZZEMBLER 22 terutama Amanda dan Rafiqha atas pertemanan dan kebaikan yang diberikan selama menempuh perjalanan studi penulis.
7. Segenap keluarga besar AIMP TRG terutama Kak Syawal, Kak Syahid, Kak Sofyan, Mutiah, serta Pak Bewagabriel, dan Tim Sampah Sungai atas bantuan dan semangat yang diberikan sehingga penelitian penulis dapat terlaksana dengan baik.
8. Pengurus Departemen Kemuslimahan GKM LD Al-Muhandis terutama Kak Izza yang selalu mengajak penulis kepada kebaikan dan menjadi arti hidup baru bagi penulis.
9. JKT48 yang terus menemani dan menyemangati penulis untuk pantang menyerah dalam menggapai impian.
10. Semua pihak yang tidak dapat penulis sebutkan satu per satu, yang telah memberikan kontribusi dalam penyelesaian skripsi ini.

Penulis menyadari bahwa skripsi ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi perbaikan di masa yang akan datang. Semoga skripsi ini dapat memberikan manfaat bagi perkembangan ilmu pengetahuan dan teknologi, khususnya dalam bidang pemantauan lingkungan berbasis kecerdasan buatan.

Makassar, 5 Juni 2026

WINATI MUTMAINNAH NURDIN
NIM D121221014

ABSTRAK

WINATI MUTMAINNAH NURDIN. **Sistem Deteksi Sampah di Permukaan Sungai Menggunakan Metode YOLO11 dan YOLOv8 Terintegrasi GPS** (dibimbing oleh Prof. Dr. Ir. Indrabayu, S.T., M.T., M.Bus.Sys., IPM., ASEAN.Eng).

Latar belakang. Pencemaran sampah di sungai merupakan permasalahan lingkungan yang serius. Metode pemantauan konvensional melalui inspeksi manual memiliki keterbatasan dalam hal cakupan area dan efisiensi. **Tujuan.** Penelitian ini bertujuan untuk membangun sistem deteksi sampah di permukaan sungai menggunakan model YOLOv8 dan YOLO11 pada citra video drone, serta memetakan lokasi sampah berdasarkan integrasi dengan koordinat GPS yang ditampilkan pada antarmuka web berbasis peta. **Metode.** Data dikumpulkan menggunakan drone di Sungai Tallo, Kota Makassar. Penelitian dilakukan dengan mengkurasi dataset menjadi 500 citra yang dibagi rasio 70:20:10 untuk data latih, validasi, dan uji. Dilakukan perbandingan model YOLOv8n, YOLOv8s, YOLO11n, dan YOLO11s pada skenario 100 dan 200 epoch. Sistem dibangun dalam *website full-stack* dengan FastAPI sebagai *backend*, *Next.js* sebagai *frontend*, dan Supabase sebagai database. **Hasil.** Kurasi dataset meningkatkan nilai mAP50 terbaik dengan peningkatan sebesar 0,0255. Pada pengujian akhir, model terbaik adalah YOLO11s dengan 200 epoch yang memperoleh mAP50 sebesar 0,7978, Precision sebesar 0,7650, dan Recall sebesar 0,7623. Sementara model terbaik pada keluarga YOLOv8 adalah YOLOv8s dengan 100 epoch yang memperoleh mAP50 sebesar 0,7919, Precision sebesar 0,7504, dan Recall sebesar 0,7415. **Kesimpulan.** Sistem yang dikembangkan berhasil mengimplementasikan model YOLO11 dan YOLOv8 untuk mendeteksi sampah di permukaan sungai serta mengintegrasikan hasil deteksi dengan data GPS drone sehingga lokasi sampah dapat dipetakan pada web.

Kata kunci: visi komputer, deteksi objek, *drone*, GPS, sampah sungai, YOLO11, YOLOv8.

ABSTRACT

WINATI MUTMAINNAH NURDIN. **River Surface Waste Detection System Using YOLO11 and YOLOv8 Integrated with GPS** (supervised by Prof. Dr. Ir. Indrabayu, S.T., M.T., M.Bus.Sys., IPM., ASEAN.Eng).

Background. Waste pollution in rivers is a serious environmental problem. Conventional monitoring methods through manual inspection are limited in area coverage and efficiency. **Objective.** This study aims to develop a river surface waste detection system using YOLOv8 and YOLO11 models on drone video imagery, and to map waste locations based on GPS coordinate integration displayed through a web-based map interface. **Methods.** Data were collected using a drone at the Tallo River, Makassar City. The study was conducted by curating the dataset into 500 images, which were divided into training, validation, and testing sets with a ratio of 70:20:10. Four model variants, namely YOLOv8n, YOLOv8s, YOLO11n, and YOLO11s, were compared under 100-epoch and 200-epoch training scenarios. The system was developed as a full-stack web application using FastAPI as the backend, Next.js as the frontend, and Supabase as the database. **Results.** Dataset curation improved the best mAP50 score by 0.0255. In the final evaluation, the best-performing model was YOLO11s trained for 200 epochs, achieving an mAP50 of 0.7978, Precision of 0.7650, and Recall of 0.7623. Meanwhile, the best-performing model within the YOLOv8 family was YOLOv8s trained for 100 epochs, achieving an mAP50 of 0.7919, Precision of 0.7504, and Recall of 0.7415. **Conclusion.** The developed system successfully implemented YOLO11 and YOLOv8 models for river surface waste detection and integrated the detection results with drone GPS data, enabling waste locations to be mapped and visualized through a web-based platform.

Keywords: computer vision, drone, GPS, object detection, river waste, YOLO11, YOLOv8.

DAFTAR ISI

UCAPAN TERIMA KASIH	i
ABSTRAK	iii
ABSTRACT	iv
DAFTAR ISI	v
DAFTAR TABEL	viii
DAFTAR GAMBAR	ix
DAFTAR LAMPIRAN	xi
DAFTAR ISTILAH/SINGKATAN	xii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Landasan Teori	2
1.2.1 Sampah	2
1.2.2 Sungai	2
1.2.3 <i>Computer vision</i>	3
1.2.4 Deteksi objek	3
1.2.5 YOLO	4
1.2.6 YOLOv8	5
1.2.7 YOLO11	6
1.2.8 <i>Unmanned Aerial Vehicle (UAV)</i>	8
1.2.9 <i>Global Positioning System (GPS)</i>	8
1.2.10 <i>Transfer learning</i>	10
1.2.11 Metrik evaluasi deteksi objek	10
1.3 Rumusan Masalah	12
1.4 Tujuan	12
1.5 Manfaat	12
1.6 Ruang Lingkup Penelitian	13
BAB II METODE PENELITIAN	14
2.1 Tempat dan Waktu Penelitian	14
2.2 Instrumen Penelitian	17

2.3 Tahapan Penelitian	18
2.4 Teknik Pengambilan Dataset.....	18
2.5 Perancangan dan Implementasi Sistem.....	19
2.5.1 Input data.....	20
2.5.2 Anotasi data.....	23
2.5.3 <i>Split data</i>	25
2.5.4 <i>Preprocessing data</i>	25
2.5.5 <i>Training model</i>	28
2.5.6 <i>Testing model</i>	29
2.5.7 Pemilihan model terbaik (best model.pt)	29
2.5.8 Implementasi model ke website sistem	30
2.6 Pembuatan Sistem	30
2.6.1 <i>Use case diagram</i>	30
2.6.2 <i>Activity diagram</i>	31
2.6.3 <i>Sequence diagram</i>	32
2.6.4 Usulan sistem website	35
BAB III HASIL DAN PEMBAHASAN	36
3.1 Dataset Awal.....	36
3.1.1 Komposisi dataset awal	36
3.1.2 Hasil pelatihan pada dataset awal.....	36
3.1.3 Kurasi dataset awal.....	37
3.2 Dataset Revisi.....	38
3.2.1 Hasil pelatihan pada dataset revisi.....	38
3.2.2 Perbandingan performa sebelum dan sesudah kurasi	39
3.3.2 Analisis pengaruh varian model dan epoch	40
3.4 Implementasi Sistem	41
3.4.1 Halaman utama.....	43
3.4.2 Halaman hasil deteksi.....	44
3.4.3 Halaman peta.....	46
3.5 Pengujian Integrasi GPS	48
3.5.1 Pengujian pembacaan data GPS dari file SRT	48

3.5.2 Pengujian sinkronisasi video dan data GPS	49
3.5.3 Pengujian visualisasi lokasi deteksi pada peta	49
BAB IV KESIMPULAN	51
4.1 Kesimpulan.....	51
4.2 Saran.....	51
DAFTAR PUSTAKA.....	53
LAMPIRAN.....	55

DAFTAR TABEL

Tabel 1. Distribusi Lokasi dari Dataset.....	24
Tabel 2. Pembagian Dataset.....	25
Tabel 3. Hasil Pengujian Model pada Dataset Awal.....	35
Tabel 4. Hasil Pengujian Model pada Dataset Revisi.....	37
Tabel 5. Hasil Pengujian Perbandingan Model Final.....	38
Tabel 6. Pengujian Fitur GPS	46

DAFTAR GAMBAR

Gambar 1. Gambaran Umum Cara Kerja YOLO	19
Gambar 2. Arsitektur YOLOv8.....	21
Gambar 3. Arsitektur YOLO11	22
Gambar 4. Ilustrasi Latitude dan Longitude	24
Gambar 5. Ilustrasi Cara Kerja GPS	24
Gambar 6. Confusion Matrix.....	26
Gambar 7. Rumus IOU	27
Gambar 8. Ilustrasi IOU	27
Gambar 9. Aliran Sungai Tallo di sekitar kawasan Inspeksi Kanal	29
Gambar 10. Aliran Sungai Tallo di sekitar Jalan Tol Insinyur Sutami	30
Gambar 11. Aliran Sungai Tallo di dekat Pelabuhan Paotere	30
Gambar 12. Danau UNHAS	30
Gambar 13. Kanal Jongaya.....	31
Gambar 14. Laboratorium Artificial Intelligence, Departemen Teknik Informatika, Fakultas Teknik, Universitas Hasanuddin	31
Gambar 15. Diagram Tahapan Penelitian.....	33
Gambar 16. Skenario Pengambilan Data	34
Gambar 17. Alur Perancangan Sistem.....	35
Gambar 18. Contoh Dataset Lokasi Sungai Tallo Kawasan Inspeksi Kanal.....	36
Gambar 19. Contoh Dataset Lokasi Sungai Tallo Kawasan Tol Ir. Sutami.....	36
Gambar 20. Contoh Dataset Lokasi Danau UNHAS	37
Gambar 21. Contoh Dataset Lokasi Kanal Jongaya.....	37
Gambar 22. Contoh Dataset Lokasi Sungai Tallo Kawasan Pelabuhan Paotere	38
Gambar 23. Contoh Hasil Anotasi Dataset Lokasi Sungai Tallo Kawasan Inspeksi Kanal.....	38
Gambar 24. Contoh Hasil Anotasi Dataset Lokasi Sungai Tallo Kawasan Tol Ir. Sutami	38
Gambar 25. Contoh Hasil Anotasi Dataset Lokasi Danau UNHAS	39
Gambar 26. Contoh Hasil Anotasi Dataset Lokasi Kanal Jongaya	39
Gambar 27. Contoh Hasil Resize Citra: (a) Sebelum Resize;	40
Gambar 28. Teknik Flip Horizontal: (a) Citra Asli;.....	41
Gambar 29. Teknik Rotasi 90°: (a) Citra Asli; (b) Rotasi 90°;.....	41
Gambar 30. Teknik Rotasi Sudut -15° hingga +15°: (a) Citra Asli;.....	41
Gambar 31. Teknik Shear ±15°: (a) Citra Asli; (b) Shear 15°, 15°;	42
Gambar 32. Use Case Diagram Sistem	44
Gambar 33. Activity Diagram Sistem	45
Gambar 34. Sequence Diagram Sistem	46
Gambar 35. Tampilan Halaman Utama tanpa Histori	54
Gambar 36. Tampilan Halaman Utama dengan Histori.....	54
Gambar 37. Tampilan Progres Deteksi	55
Gambar 38. Tampilan Progres Berhasil (Summary).....	55
Gambar 39. Tampilan Progres Berhasil (Map)	55
Gambar 40. Tampilan Progres Berhasil (Video)	56
Gambar 41. Tampilan Progres Berhasil (Gallery).....	56

Gambar 42. Tampilan Halaman Peta Polos.....	57
Gambar 43. Tampilan Halaman Peta dengan Marker.	57
Gambar 44. Tampilan Halaman Peta dengan Informasi Detail Marker.....	58
Gambar 45. Contoh Data GPS pada File SRT	58
Gambar 46. Hasil Ekstraksi Data GPS oleh Sistem Lokal.....	59

DAFTAR LAMPIRAN

Lampiran 1. Dataset	65
Lampiran 2. Source Code Website	66
Lampiran 3. Website Sistem.....	67

DAFTAR ISTILAH/SINGKATAN

Istilah/ Singkatan	Keterangan
AI	<i>Artificial Intelligence</i> , cabang ilmu komputer yang memungkinkan sistem meniru kecerdasan manusia.
API	<i>Application Programming Interface</i> , antarmuka yang memungkinkan komunikasi antar aplikasi.
ASGI	<i>Asynchronous Server Gateway Interface</i> , standar komunikasi aplikasi Python modern.
<i>Bounding Box</i>	Kotak pembatas yang digunakan untuk menunjukkan lokasi objek hasil deteksi.
CNN	<i>Convolutional Neural Network</i> , arsitektur jaringan saraf yang umum digunakan pada pengolahan citra.
COCO	<i>Common Objects in Context</i> , dataset standar yang digunakan untuk pelatihan awal model deteksi objek.
<i>Confidence Score</i>	Nilai kepercayaan model terhadap hasil deteksi objek.
<i>Deep Learning</i>	Metode machine learning berbasis jaringan saraf berlapis banyak.
FastAPI	<i>Framework Python</i> yang digunakan untuk membangun layanan backend berbasis API.
FPS	<i>Frames Per Second</i> , jumlah frame yang ditampilkan atau direkam setiap detik pada video.
GPS	<i>Global Positioning System</i> , sistem navigasi satelit untuk menentukan posisi geografis.
IoU	<i>Intersection over Union</i> , metrik untuk mengukur tingkat kesesuaian antara bounding box prediksi dan <i>ground truth</i> .
Roboflow	Platform yang digunakan untuk anotasi, augmentasi, dan manajemen <i>dataset computer vision</i> .
SPPF	<i>Spatial Pyramid Pooling Fast</i> , modul pada YOLO untuk memperluas <i>receptive field</i> secara efisien.
SRT	<i>SubRip Subtitle</i> , format file yang digunakan untuk menyimpan data GPS dan <i>timestamp</i> dari <i>drone DJI</i> .

BAB I PENDAHULUAN

1.1 Latar Belakang

Perkembangan sistem bantuan pengemudi atau advanced driver assistance systems (ADAS) menunjukkan bahwa teknologi ini semakin diposisikan sebagai komponen penting dalam peningkatan keselamatan berkendara. Temuan terbaru menunjukkan bahwa penerapan ADAS berpotensi menurunkan angka kecelakaan berat serta mendukung agenda keselamatan jalan raya secara lebih luas, khususnya ketika fitur-fitur bantu berkendara mulai diadopsi secara lebih merata pada kendaraan modern (Costa et al., 2025). Sejalan dengan itu, kajian terhadap ADAS juga menunjukkan bahwa pengguna cenderung menilai sistem ini mampu meningkatkan rasa aman dan kenyamanan berkendara, meskipun masih dibutuhkan penyempurnaan berkelanjutan agar manfaatnya benar-benar optimal dalam konteks keselamatan (Neumann, 2024). Kondisi tersebut menegaskan bahwa kebutuhan akan sistem bantuan pengemudi yang andal bukan lagi sekadar pilihan teknologi, melainkan kebutuhan nyata dalam upaya menekan risiko kecelakaan di jalan.

Dalam konteks kendaraan cerdas, kemampuan sistem untuk memahami lingkungan secara cepat dan akurat menjadi prasyarat utama. ADAS sangat bergantung pada pemrosesan video dan citra secara terus-menerus agar kendaraan dapat mengenali objek penting, membaca situasi lalu lintas, serta merespons potensi bahaya secara tepat waktu. Studi terkini menekankan bahwa penguatan ADAS melalui computer vision dan pendekatan machine learning memang meningkatkan kapabilitas deteksi, tetapi sekaligus menuntut proses komputasi yang lebih besar dan berkelanjutan (Shah et al., 2025). Pada saat yang sama, implementasi berbasis perangkat embedded edge untuk deteksi objek dan pembentukan informasi lingkungan menunjukkan bahwa pengolahan data visual secara real-time merupakan aspek mendasar dalam sistem kendaraan yang aman dan responsif (Choi et al., 2024). Oleh karena itu, real-time processing bukan hanya karakteristik teknis, melainkan inti dari keandalan sistem keselamatan pada kendaraan cerdas.

Persoalan utama muncul ketika kebutuhan pemrosesan real-time berhadapan dengan tuntutan latensi yang sangat rendah. Pada sistem berbasis edge, keterlambatan sekecil apa pun dapat mengurangi kualitas keputusan dan menurunkan efektivitas peringatan dini, terutama ketika beban inferensi berjalan secara dinamis di lingkungan jaringan yang berubah-ubah. Kajian mengenai inference serving dengan end-to-end latency SLOs menunjukkan bahwa ketepatan waktu penyajian inferensi sama pentingnya dengan akurasi model, khususnya pada jaringan edge yang bersifat dinamis (Nigade et al., 2024). Di sisi lain, penelitian tentang penempatan server edge menegaskan bahwa pengaturan sumber daya yang tidak tepat dapat memperbesar latensi transmisi dan memperberat beban kerja sistem, sehingga mengganggu pencapaian kinerja waktu nyata yang dibutuhkan aplikasi kritis (Zhang et al., 2024). Dengan demikian, tantangan utama pada ADAS bukan hanya menghasilkan keluaran yang benar, tetapi juga memastikan keluaran tersebut hadir dalam batas waktu yang aman untuk pengambilan keputusan berkendara.

Tantangan tersebut menjadi semakin kompleks ketika sistem dijalankan pada

perangkat embedded dengan sumber daya terbatas. Optimasi deployment pada platform embedded heterogen diperlukan agar kebutuhan frame rate, akurasi, dan efisiensi energi tetap terpenuhi (Ruiz-Barroso et al., 2025). Platform seperti Jetson Orin Nano menawarkan kemampuan komputasi yang memadai untuk kelas perangkat edge berbiaya rendah, namun tetap berada dalam batasan konsumsi daya, suhu operasi, dan kapasitas pemrosesan yang jauh lebih ketat dibandingkan platform edge kelas atas (mis. Jetson AGX Orin) maupun server kelas pusat. Kajian terhadap perangkat embedded menunjukkan bahwa pemrosesan video real-time selalu menuntut kompromi antara kecepatan, efisiensi energi, dan stabilitas kinerja, terutama ketika sistem harus mempertahankan hasil yang konsisten pada beban kerja yang berubah-ubah (Suder et al., 2023). Dalam konteks aplikasi keselamatan, kompromi semacam ini menjadi sangat penting karena keterbatasan sumber daya dapat langsung berpengaruh pada keterlambatan deteksi, penurunan laju pemrosesan, dan berkurangnya kemampuan sistem dalam mempertahankan performa secara berkelanjutan.

Selain beban komputasi, sistem ADAS juga menuntut karakteristik deterministik dan stabil karena bersifat safety-critical. Setiap proses harus selesai sesuai tenggat waktu agar fungsi peringatan, deteksi, dan respons dapat berjalan secara konsisten di bawah kondisi operasional yang berubah cepat. Studi terbaru menunjukkan bahwa penjadwalan tugas pada ADAS memiliki tingkat kepekaan yang tinggi terhadap deadline, dan kegagalan memenuhi batas waktu dapat berimplikasi pada kegagalan sistem secara fungsional (Seyfipoor et al., 2026). Hal ini menegaskan bahwa aplikasi keselamatan memerlukan arsitektur yang tidak hanya cepat, tetapi juga stabil, terprediksi, dan efisien dalam mengelola prioritas pemrosesan. Dengan kata lain, keandalan ADAS tidak dapat dilepaskan dari kemampuan sistem untuk menjaga konsistensi waktu proses dalam kondisi beban tinggi — baik beban tersebut berasal dari tahap deteksi objek (inferensi model dan post-processing NMS-nya) maupun dari tahap asosiasi objek antar-frame (tracking) yang menjaga identitas objek tetap konsisten sepanjang waktu.

Berdasarkan uraian tersebut, terlihat bahwa penelitian mengenai optimasi real-time pipeline pada platform embedded menjadi relevan dan mendesak untuk dilakukan, khususnya pada konteks ADAS berbasis Jetson Orin Nano. Permasalahan yang dihadapi bukan semata-mata pada kemampuan mendeteksi objek, melainkan pada bagaimana sistem dapat mempertahankan kecepatan, kestabilan, dan efisiensi pemrosesan secara simultan dalam skenario operasi yang menuntut respons segera — pada setiap tahap pipeline, mulai dari inferensi deteksi, post-processing (NMS), hingga tracking. Oleh karena itu, penelitian ini diarahkan untuk memahami dan menganalisis faktor-faktor yang memengaruhi kinerja real-time pipeline agar sistem ADAS dapat memenuhi kebutuhan keselamatan berkendara secara lebih andal.

1.2 Landasan Teori

1.2.1 ADAS dan *Perception Layer* pada Kendaraan Cerdas

Advanced Driver Assistance Systems (ADAS) adalah kumpulan teknologi sistem elektronik terintegrasi yang dirancang untuk membantu pengemudi dalam proses berkendara dan memarkir kendaraan guna meningkatkan keselamatan dan kenyamanan. Sistem ADAS modern sangat bergantung pada pemahaman lingkungan

sekitar secara real-time untuk mendeteksi potensi bahaya, seperti keberadaan kendaraan lain, pejalan kaki, atau infrastruktur jalan (Neumann, 2024).

Secara arsitektural, sistem ADAS terdiri dari tiga lapisan utama: perception layer (lapisan persepsi), planning layer (lapisan perencanaan), dan control layer (lapisan kendali) (Shah et al., 2025). Penelitian ini secara khusus berfokus pada perception layer, yaitu tahapan di mana data mentah dari sensor (dalam hal ini, citra video dari kamera) diekstraksi menjadi informasi semantik berwujud deteksi dan pelacakan objek. Pada lapisan ini, keandalan sistem tidak hanya diukur dari seberapa akurat objek dikenali, melainkan juga seberapa cepat informasi tersebut dapat disajikan ke lapisan perencanaan, karena keterlambatan sekecil apapun dapat mengurangi waktu respons pengereman darurat atau peringatan dini.

1.2.2 Konsep *Edge Computing* dan Analisis Video *Real-Time*

Pemrosesan *perception layer* pada kendaraan otonom dan ADAS umumnya diimplementasikan menggunakan paradigma *edge computing*, di mana komputasi data dilakukan secara lokal di perangkat (*on-device*) yang berada sedekat mungkin dengan sumber data sensor (Choi et al., 2024). Pendekatan ini meniadakan ketergantungan pada koneksi jaringan *cloud* yang memiliki latensi transmisi dinamis dan rentan terputus (Zhang et al., 2024).

Namun, implementasi *Edge AI* memiliki batasan mendasar terkait anggaran sumber daya (*resource constraints*). Perangkat *embedded edge* (seperti keluarga NVIDIA Jetson) beroperasi dengan ketersediaan memori, kapabilitas komputasi (CPU/GPU), dan konsumsi daya (*thermal design power*) yang sangat terbatas dibandingkan server pusat (Suder et al., 2023). Oleh karena itu, *real-time video analytics* pada *edge device* selalu menuntut kompromi yang optimal antara akurasi model, efisiensi energi, dan kecepatan pemrosesan. Kinerja *real-time* dalam konteks ini didefinisikan sebagai kemampuan sistem dalam mengeksekusi *pipeline* secara deterministik guna memenuhi tenggat waktu (*deadline*) laju bingkai (misalnya 30 FPS) secara terus-menerus tanpa penumpukan antrian pemrosesan (Seyfipour et al., 2026).

1.2.3 NVIDIA DeepStream SDK dan *Pipeline* Inferensi

NVIDIA DeepStream adalah *Software Development Kit* (SDK) yang dibangun di atas kerangka kerja multimedia GStreamer, dirancang khusus untuk membangun *pipeline* analitik video bertenaga kecerdasan buatan (AI) secara *real-time* dengan memanfaatkan akselerasi perangkat keras NVIDIA.

Karakteristik kunci DeepStream yang membedakannya dari *pipeline* konvensional (seperti integrasi manual berbasis OpenCV) adalah penggunaan arsitektur memori *zero-copy* berbasis NVIDIA Memory Manager (NVMM). Pada *pipeline* standar, *frame* gambar sering disalin secara berulang antara memori CPU (*host*) dan GPU (*device*) pada setiap tahapan (decode, prapemrosesan, inferensi, *post-processing*). NVMM mengeliminasi hambatan (*bottleneck*) ini dengan menahan *buffer frame* video secara persisten di dalam memori GPU dari awal hingga akhir *pipeline*.

1.2.4 Deteksi Objek *Real-Time* dan Arsitektur YOLO

You Only Look Once (YOLO) merupakan keluarga arsitektur jaringan saraf tiruan untuk deteksi objek berbasis pendekatan *single-stage detector*. Berbeda dengan metode *two-stage* (seperti *Faster R-CNN*) yang memerlukan tahapan proposal region sebelum klasifikasi, YOLO memprediksi koordinat *bounding box* dan probabilitas kelas secara bersamaan dalam satu kali eksekusi (*forward pass*) jaringan penuh. Karakteristik ini membuat YOLO sangat ideal untuk pemrosesan *real-time* di lingkungan ADAS (Ayachi et al., 2025; Dhatrika et al., 2025).

Penelitian ini mengevaluasi berbagai generasi kelas ringan (*nano/tiny*) dari YOLO, seperti YOLOv8, YOLOv9, YOLOv10, dan YOLOv26. Perbedaan struktural penting di antara generasi ini terletak pada strategi *post-processing*. Generasi awal hingga v9 menghasilkan banyak prediksi kotak redundan di sekitar objek tunggal, sehingga mewajibkan penggunaan filter *Non-Maximum Suppression* (NMS). Sebaliknya, arsitektur yang lebih modern (seperti YOLOv10n dan YOLOv26n) telah mengintegrasikan metode optimasi struktural (*consistent dual assignments*) selama pelatihan, menjadikannya model *NMS-free* yang tidak lagi memerlukan *post-processing* heuristik eksternal, sehingga berpotensi menurunkan latensi inferensi.

1.2.5 *Post-Processing* Deteksi: *Bounding Box*, IoU, dan NMS

Keluaran mentah dari arsitektur deteksi objek standar (*non-NMS-free*) terdiri dari sekumpulan *bounding box* kandidat. Setiap kandidat direpresentasikan oleh parameter koordinat (x, y, w, h) dan sebuah nilai keyakinan (*confidence score*) yang menyatakan probabilitas keberadaan objek berserta kelasnya.

Untuk mengevaluasi tingkat tumpang-tindih antar kandidat kotak, digunakan metrik *Intersection over Union* (IoU), yang diformulasikan sebagai rasio luas irisan terhadap luas gabungan dari dua *bounding box* (A dan B):

$$IoU = \frac{\text{Area}(A \cap B)}{\text{Area}(A \cup B)}$$

Non-Maximum Suppression (NMS) adalah algoritma pascapemrosesan yang menyaring kandidat berlebih tersebut. Proses NMS standar mengurutkan seluruh *box* berdasarkan *confidence score* tertinggi, mempertahankan kotak dengan skor tertinggi, lalu menghapus kotak-kotak lain yang memiliki nilai IoU dengan kotak tertinggi melampaui suatu nilai ambang batas (*threshold*) tertentu (misalnya $IoU > 0.45$).

Implementasi NMS tradisional (seperti GreedyNMS) bersifat sekuensial dan umumnya dieksekusi oleh CPU (*host*), yang seringkali menjadi hambatan latensi pada pemrosesan *edge* akibat proses perpindahan data dan interupsi siklus (Chen et al., 2022; Oro et al., 2022). Untuk menanggulangi hal ini, *plugin EfficientNMS_TRT* mengintegrasikan tahapan NMS secara langsung ke dalam graf komputasi TensorRT. Pendekatan ini memungkinkan perbandingan IoU dan penyaringan objek dilakukan secara paralel penuh (*fully parallelized*) di dalam *kernel* GPU, menghilangkan *bottleneck* transfer memori ke CPU dan mempercepat penyajian inferensi (Yang et al., 2025).

1.2.6 Konsep Dasar *Multi-Object Tracking* (MOT)

Deteksi objek berbasis *frame* tunggal rentan terhadap ketidakstabilan visual, seperti *flicker* akibat oklusi sementara, perubahan pencahayaan, atau kegagalan detektor secara sesaat. *Multi-Object Tracking* (MOT) mengatasi masalah ini dengan mengasosiasikan deteksi pada *frame* saat ini dengan lintasan historis objek dari *frame-frame* sebelumnya, memberikan identitas unik (ID) yang konsisten.

Algoritma MOT paling efisien berakar pada paradigma Tracking-by-Detection yang utamanya menggunakan dua komponen matematis:

1. **Kalman Filter:** Sebuah algoritma estimasi rekursif yang memprediksi keadaan kinematik objek di masa depan (posisi dan kecepatan) berdasarkan pengukuran beruntun yang mengandung derau (*noise*).
2. **Hungarian Algorithm:** Algoritma optimasi kombinatorial untuk memecahkan masalah penugasan (*assignment problem*). Dalam MOT, algoritma ini digunakan untuk mencocokkan (*bipartite matching*) prediksi lokasi *Kalman Filter* dengan lokasi *bounding box* deteksi baru, biasanya menggunakan matriks jarak berbasis IoU atau jarak Euclidean.

1.2.7 Algoritma *Tracking* pada DeepStream

DeepStream menyediakan elemen nvtracker yang mendukung berbagai profil pelacakan. Penelitian ini mengevaluasi dua algoritma pelacakan dengan pendekatan desain yang mewakili kedua ujung spektrum beban komputasi:

- **NvSORT (*Simple Online and Realtime Tracking*):** Pelacak berbasis *motion-only* murni yang hanya mengandalkan *Kalman Filter* dan *Hungarian Algorithm* pada properti koordinat spasial tanpa mengekstraksi atau memproses data piksel gambar sama sekali. Hal ini membuatnya beroperasi sangat cepat dan efisien terhadap utilisasi CPU/GPU, dengan kompromi berkurangnya kemampuan membedakan objek saat terjadi oklusi silang yang kompleks (Shin & Li, 2023).
- **NvDCF (*Discriminative Correlation Filter*):** Pelacak hibrida *feature-based* yang mempelajari ciri visual khusus (*learned visual features*) dari setiap target secara *real-time*. NvDCF memproses area piksel dalam *bounding box* untuk membuat filter korelasi, yang memungkinkannya melacak objek saat detektor gagal atau tertutup sebagian secara tangguh. Konsekuensinya, NvDCF membutuhkan beban komputasi yang jauh lebih berat karena mengeksekusi ekstraksi fitur pada setiap objek di setiap *frame*.

1.2.8 Akselerasi Inferensi: TensorRT dan Kuantisasi Presisi

NVIDIA TensorRT adalah *runtime* optimasi *deep learning* yang mengkompilasi model jaringan saraf (seperti arsitektur ONNX) menjadi *engine* yang dieksekusi secara spesifik untuk mikroarsitektur perangkat keras target (GPU Ampere). TensorRT melakukan fusi lapisan (*layer fusion*), kalibrasi presisi, dan pemilihan kernel (*kernel auto-tuning*).

Kuantisasi adalah teknik mereduksi kedalaman bit yang digunakan untuk merepresentasikan bobot dan aktivasi jaringan. Sementara pelatihan model konvensional dilakukan dalam presisi *Floating-Point 32-bit* (FP32), inferensi pada perangkat *edge* dioptimalkan menggunakan presisi lebih rendah. Presisi *Floating-Point 16-bit* (FP16) dapat mempercepat kalkulasi berkat penggunaan unit *Tensor Core* pada arsitektur GPU dengan penurunan akurasi yang marjinal.

1.2.9 Metrik Evaluasi Kinerja *Real-Time Pipeline*

Evaluasi kinerja pipeline pada komputasi lokal *edge* didasarkan pada efisiensi komputasi, kelancaran eksekusi, serta mempertahankan akurasi dasar (Nigade et al., 2024). Metrik yang digunakan meliputi:

A. *Throughput* (FPS)

Laju bingkai per detik (*Frames Per Second*) mengukur agregat volume pemrosesan. Diformulasikan secara konseptual dari total latensi inversi rata-rata sistem per *frame*:

$$FPS = \frac{1}{\text{Rata-rata } t_{e2e} \text{ (detik)}}$$

di mana sistem dikategorikan *real-time* jika nilai laju pemrosesan stabil pada target aplikasi kritis (misalnya minimum 30 FPS).

B. Latensi dan Persentil (p95) Latensi adalah durasi tempuh pemrosesan. Pada penelitian *pipeline*, dievaluasi latensi *end-to-end* (t_{e2e}) yang diukur dari titik bingkai masuk *nvstreammux* hingga keluar di blok antarmuka layar, dan dirincikan secara per-komponen (contoh: *Lat_Infer_ms*, *Lat_Tracker_ms*). Dalam sistem *safety-critical* seperti ADAS, rerata (mean) rentan menyembunyikan efek variabilitas ekstrem (*jitter*). Oleh karena itu, metrik menggunakan persentil ke-95 (p95) yang mengindikasikan bahwa 95% dari seluruh siklus *frame* diselesaikan dalam durasi kurang dari atau sama dengan nilai latensi tersebut.

C. Akurasi Deteksi Untuk memastikan optimasi presisi komputasi (FP16) dan implementasi NMS tidak merusak fungsionalitas pengenalan (*sanity-check*), akurasi diukur berdasar matrik kebingungan (*confusion matrix*):

- *Precision* (P): Proporsi deteksi yang relevan (benar).

$$P = \frac{TP}{TP + FP}$$

- *Recall* (R): Proporsi target asli yang berhasil ditemukan.

$$R = \frac{TP}{TP + FN}$$

- *mean Average Precision* (mAP): Merupakan luas area di bawah kurva *Precision-Recall* rata-rata untuk seluruh kelas objek, dihitung melintasi batas ambang batas ($IoU = 0.5$ untuk mAP50, dan rentang $IoU = 0.5: 0.95$ untuk mAP50-95).

1.3 Rumusan Masalah

Rumusan masalah dari penelitian ini adalah sebagai berikut:

1. Bagaimana kinerja real-time pipeline Nvidia DeepStream pada proses deteksi kendaraan di Jetson Orin Nano?

2. Sejauh mana penerapan NMS paralel berbasis TensorRT plugin (EfficientNMS) dapat meningkatkan efisiensi pipeline deteksi kendaraan dibandingkan konfigurasi standar?
3. Sejauh mana pemilihan algoritma tracking (NvDCF vs. NvSORT) memengaruhi efisiensi komputasi real-time pipeline dibandingkan konfigurasi baseline?

1.4 Tujuan

Penelitian ini memiliki tujuan antara lain:

1. Menganalisis kinerja *real-time pipeline* Nvidia DeepStream pada deteksi kendaraan di Jetson Orin Nano.
2. Mengevaluasi pengaruh optimasi *pipeline*, khususnya penerapan NMS paralel berbasis TensorRT plugin (EfficientNMS), terhadap peningkatan performa pemrosesan *real-time*.
3. Mengevaluasi pengaruh pemilihan algoritma *tracking* terhadap efisiensi komputasi *real-time pipeline* dibandingkan konfigurasi *baseline*.

1.5 Manfaat

Manfaat Teoritis — Menjadi referensi ilmiah mengenai optimasi *pipeline* pemrosesan video *real-time* pada sistem *edge* untuk aplikasi ADAS, khususnya dalam penerapan Nvidia DeepStream, NMS paralel berbasis TensorRT plugin (EfficientNMS), serta evaluasi efisiensi komputasi pemilihan algoritma *tracking* pada perangkat Jetson Orin Nano.

Manfaat Praktis — Memberikan pedoman teknis bagi pengembang sistem *embedded* dan ADAS mengenai konfigurasi *pipeline* yang paling optimal untuk mencapai kinerja *real-time*, terutama dalam aspek latensi, *throughput*, dan stabilitas pemrosesan pada perangkat Jetson Orin Nano.

Manfaat Institusional — Berkontribusi dalam pengembangan riset terapan di bidang Visi Komputer, *Edge AI*, dan *Embedded Systems*, serta memperkuat kapasitas institusi dalam kajian pemanfaatan teknologi akselerasi perangkat keras NVIDIA pada sistem cerdas berbasis video.

1.6 Ruang Lingkup Penelitian

1. Agar penelitian lebih terarah dan fokus, maka batasan penelitian ini adalah sebagai berikut:
2. Pengujian penelitian dilakukan hanya pada perangkat Jetson Orin Nano sebagai platform utama eksperimen.
3. Peningkatan performa dianalisis hanya berdasarkan perbandingan pipeline sebelum dan sesudah optimasi menggunakan NMS paralel berbasis TensorRT plugin (EfficientNMS) dan pemilihan algoritma tracking (NvDCF vs. NvSORT).
4. Penelitian menggunakan model pre-trained sehingga tidak berfokus pada proses pelatihan maupun peningkatan akurasi model deteksi.
5. Penelitian hanya membahas pipeline pemrosesan video dan tidak mencakup integrasi dengan sistem kendaraan secara keseluruhan.
6. Evaluasi algoritma tracking dibatasi pada aspek efisiensi komputasi (FPS,

latensi per-komponen, utilisasi resource), dan *tidak mencakup evaluasi kualitas atau akurasi tracking** (mis. jumlah ID switch, MOTA/IDF1). Pembatasan ini konsisten dengan rumusan masalah ketiga yang memang dirumuskan secara eksplisit sebagai pertanyaan efisiensi komputasi (§1.2), sejalan dengan sumbu akurasi-vs-komputasi yang sudah melekat pada desain profil NvDCF dan NvSORT itu sendiri menurut dokumentasi resmi NVIDIA (lihat Bab II §2.2.6) — bukan sesuatu yang perlu diukur ulang penelitian ini agar pertanyaannya valid dijawab. Ketidaktersediaan dataset tracking* berlabel (video berurutan dengan ID objek konsisten, mis. KITTI Tracking) pada ruang lingkup penelitian ini menjadi alasan pendukung tambahan.

BAB II METODE PENELITIAN

2.1 Tempat dan Waktu Penelitian

Penelitian serta pengujian optimasi model inferensi dilaksanakan di Departemen Teknik Informatika, Universitas Hasanuddin. Keseluruhan tahapan penelitian yang meliputi persiapan lingkungan sistem, kompilasi pipeline, hingga pengambilan data performa perangkat eksekusi dilakukan pada rentang waktu Mei 2026 hingga Agustus 2026.

2.2 Instrumen Penelitian

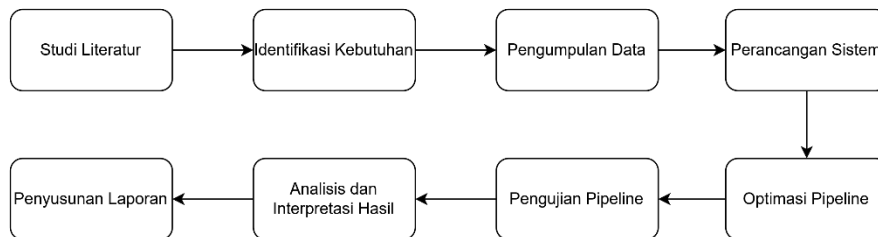
Instrumen yang digunakan dalam penelitian ini adalah sebagai berikut:

1. Perangkat lunak
 - a. Sistem Operasi Arch Linux 64bit
 - b. Sistem Operasi Windows 11 64bit
 - c. Sistem Operasi L4T (Linux for Tegra / Ubuntu Linux)
 - d. *Notebook* Kaggle GPU NVIDIA Tesla P100
 - e. *Zed Editor*
 - f. Github
 - g. *Zen Web Browser*
 - h. Microsoft Office 365
 - i. Canva
 - j. Lucidchart
 - k. Mandeley Reference Manager
 - l. NVIDIA JetPack
 - m. Nvidia Deepstream SDK
 - n. NVIDIA TensorRT
 - o. Nvidia Nsight System
2. Perangkat keras
 - a. Laptop Advan Workplus AMD Ryzen 5 6600H dengan Radeon Graphics (3.30GHz), ram 16GB, SSD 512GB,
 - b. Nvidia Jetson Orin Nano Super Board 4GB, GPU Ampere, 512 CUDA core, 16 Tensor core (generasi ke-3), hingga 625 MHz
3. Bahasa Pemrograman
 - a. C++
 - b. Python v3
 - c. Bash
4. Library Backend
 - a. Ultralytics (YOLO) oleh Tim Ultralytics
Digunakan untuk deteksi objek pada *frame* video.
 - b. Numpy oleh Travis Oliphant
Digunakan untuk komputasi numerik dan manipulasi array.
 - c. Pillow
Digunakan untuk menyimpan *snapshot frame* dan manipulasi gambar.

2.3 Tahapan Penelitian

Penelitian dilakukan dengan beberapa tahapan seperti yang ditunjukkan pada Gambar 16 berikut:

Penelitian diawali dengan ide



Gambar 16. Diagram Tahapan Penelitian

Tahapan penelitian disusun sebagai alur kerja terstruktur dari tahap studi literatur hingga pelaporan. Uraian teknis setiap tahapan adalah sebagai berikut:

1. Studi Literatur Tahap ini mengkaji penelitian terdahulu terkait sistem bantuan pengemudi (ADAS), optimasi *pipeline* pemrosesan video *real-time* pada sistem *edge*, implementasi model keluarga YOLO untuk deteksi objek, serta metode akselerasi algoritma *Non-Maximum Suppression* (NMS) dan algoritma *tracking*.
2. Identifikasi Kebutuhan Penelitian Tahap ini merumuskan kebutuhan penelitian secara sistematis untuk menetapkan permasalahan, tujuan, spesifikasi perangkat keras (Jetson Orin Nano 4GB), serta batasan penelitian.
3. Pengumpulan Data Tahap ini meliputi persiapan data yang akan dievaluasi. Data yang disiapkan mencakup penyiapan *dataset* KITTI dengan pemetaan ulang tiga kelas kustom (mobil, van, truk) untuk evaluasi akurasi, serta penyiapan berkas video pengujian untuk proses *benchmark* performa *runtime*.
4. Perancangan Sistem Tahap ini merancang arsitektur *pipeline* sistem *DeepStream* secara menyeluruh. Perancangan meliputi skema *input* video, inferensi menggunakan model YOLO (v8n, v9t, v10n, v26n).
5. Optimasi Pipeline Tahap ini mengimplementasikan dan mengoptimalkan rancangan *pipeline* menggunakan bahasa C++ dan kerangka kerja *GStreamer*. Optimasi yang dilakukan meliputi penerapan *plugin EfficientNMS-TRT* untuk mengeksekusi proses NMS secara paralel pada GPU dan konfigurasi algoritma *tracker* untuk membandingkan efisiensi komputasi.
6. Pengujian Sistem Tahap ini mengevaluasi kinerja *pipeline* pada lingkungan Jetson Orin Nano. Pengujian dilakukan pada tiga skenario utama: pengukuran *baseline* model deteksi, pengukuran performa setelah penerapan NMS paralel, dan evaluasi efisiensi komputasi antara algoritma *tracking* *NvDCF* dan *NvSORT*.
7. Analisis dan Interpretasi Hasil Tahap ini menginterpretasikan hasil evaluasi secara komprehensif melalui ekstraksi log perangkat keras (*tegrastats*) dan metrik *runtime* (*fps.csv*). Data dianalisis untuk membandingkan *throughput* (FPS), latensi *end-to-end* maupun per-komponen, utilisasi perangkat keras (CPU, GPU, RAM, daya), serta menganalisis *trade-off* antara kecepatan dan akurasi.

8. Penyusunan Laporan Tahap ini mendokumentasikan seluruh proses dan hasil penelitian secara sistematis dan terstruktur ke dalam bentuk laporan akhir.

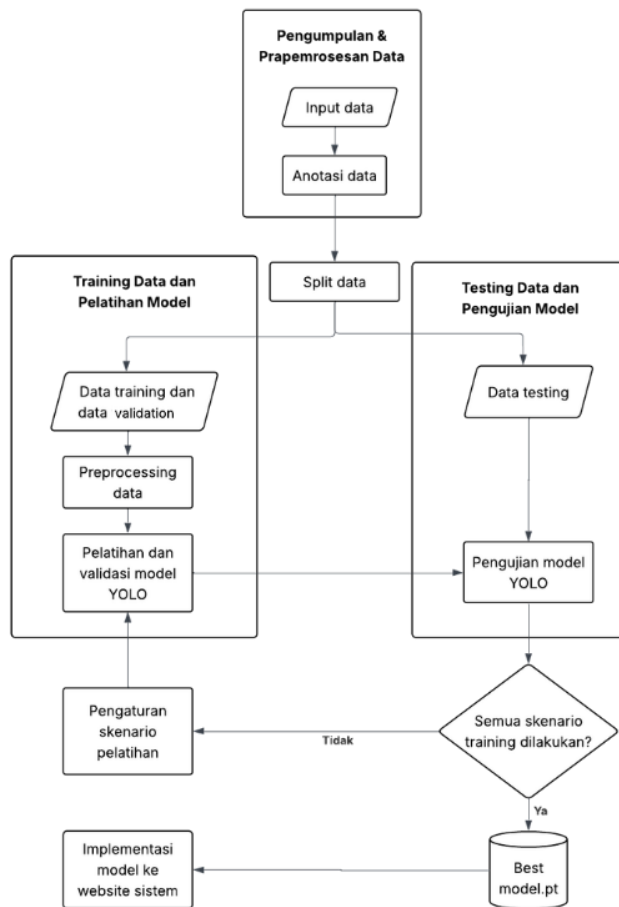
2.4 Teknik Pengambilan Dataset

Pengumpulan data dalam penelitian ini sepenuhnya menggunakan data sekunder yang bersumber dari dataset publik KITTI Vision Benchmark Suite. Pemilihan sumber data ini didasarkan pada karakteristik penelitian yang menggunakan model pre-trained dan berfokus pada optimasi performa pipeline deployment di perangkat edge, sehingga pengumpulan dataset diarahkan untuk memenuhi kebutuhan pengujian akurasi deteksi sebagai baseline proxy, bukan untuk keperluan pelatihan model dari awal. Dalam proses implementasinya, dilakukan pemetaan ulang terhadap anotasi asli KITTI menjadi tiga kelas kustom yang disesuaikan dengan tujuan penelitian, yaitu kelas car yang diberi label 0, kelas van yang diberi label 1, dan kelas truck yang diberi label 2. Proses re-mapping ini penting dilakukan mengingat anotasi bawaan KITTI memiliki kategori objek yang lebih beragam sehingga perlu disederhanakan agar selaras dengan skema klasifikasi yang digunakan dalam sistem deteksi yang dikembangkan.

Distribusi pembagian berkas citra yang digunakan dalam penelitian ini terdiri atas 5.788 citra untuk data latih dan 1.010 citra untuk data validasi. Pada subset data validasi yang berjumlah 1.010 citra tersebut, teridentifikasi total 4.722 instance objek dengan rincian distribusi kelas yang menunjukkan ketimpangan atau class imbalance yang cukup signifikan. Kelas car mendominasi dengan jumlah 4.167 instance, diikuti oleh kelas van sebanyak 405 instance, dan kelas truck sebanyak 150 instance. Ketimpangan distribusi kelas ini menjadi salah satu aspek penting yang perlu diperhatikan dalam interpretasi hasil evaluasi model, mengingat performa deteksi pada kelas dengan jumlah instance yang lebih sedikit berpotensi menghasilkan metrik yang kurang representatif dibandingkan kelas yang mendominasi dataset.

2.5 Perancangan dan Implementasi Sistem

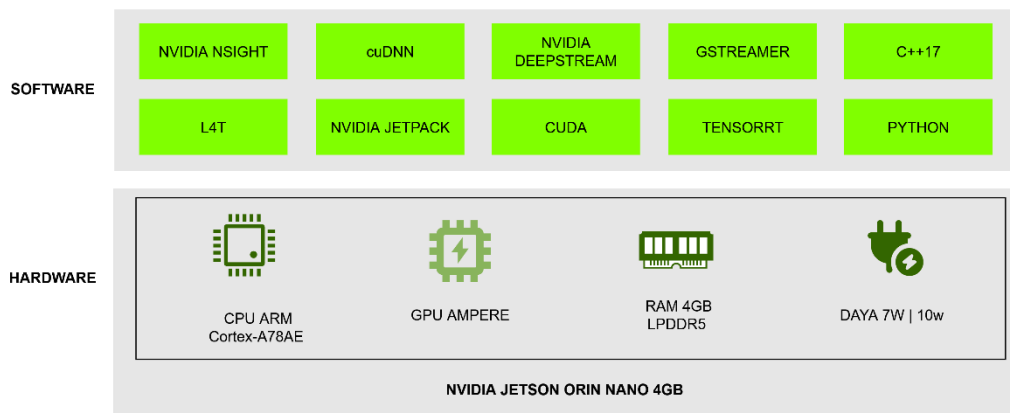
Perancangan sistem pada penelitian ini mengikuti alur sebagaimana ditunjukkan pada Gambar 18. Alur dimulai dari tahap input data hingga implementasi model terbaik ke dalam website sistem. Setelah proses anotasi dan pembagian data, dilakukan serangkaian skenario pelatihan dan pengujian model YOLO menggunakan kombinasi dataset, ukuran model, dan jumlah epoch yang berbeda. Hasil dari setiap skenario kemudian dibandingkan untuk menentukan model terbaik yang akan diimplementasikan pada website sistem.



Gambar 18. Alur Perancangan Sistem

Berdasarkan Gambar 18, setelah data dibagi (*split data*), terdapat dua jalur paralel: jalur kiri untuk proses pelatihan menggunakan *data training* dan validasi, serta jalur kanan untuk pengujian menggunakan *data testing*. Kedua jalur bermuara pada tahap evaluasi yang menentukan apakah model layak dilanjutkan ke implementasi. Berikut penjelasan setiap tahap dalam alur tersebut.

2.5.1 Konfigurasi Edge Device



Perangkat keras utama yang digunakan sebagai *edge device* dalam penelitian ini adalah NVIDIA Jetson Orin Nano varian 4GB. Perangkat ini dioperasikan pada mode daya standar 7W/10W, yang dikonfigurasi melalui indeks mode 0 (`nvpmodel -m 0`). Frekuensi *clock* perangkat juga dikunci pada tingkat maksimal menggunakan utilitas `jetson_clocks` untuk memastikan stabilitas performa selama pengujian berlangsung. Sebagai sumber masukan video, sistem ini menggunakan kamera Stereolabs ZED yang diatur untuk menghasilkan aliran video pada kecepatan 30 *Frames Per Second* (FPS).

Lingkungan perangkat lunak yang diimplementasikan pada sistem ini dibangun di atas kerangka kerja GStreamer 1.0 dan memanfaatkan NVIDIA DeepStream SDK versi 7.1. Untuk menunjang eksekusi model deep learning secara efisien, TensorRT digunakan dengan pengaturan tingkat presisi Half-Precision Floating-Point (FP16). Keseluruhan program dan perancangan pipeline diimplementasikan menggunakan bahasa pemrograman C++17.

2.5.2 Persiapan Model dan Dataset

[Gambar 2: Arsitektur model YOLO kelas nano/tiny dan ilustrasi contoh dataset KITTI]
 Persiapan data pada sistem ini memanfaatkan dataset KITTI yang telah dipetakan ulang menjadi tiga kelas objek khusus kendaraan, yaitu *car* (0), *van* (1), dan *truck* (2). Empat varian model arsitektur YOLO kelas *nano* dan *tiny* dievaluasi dalam penelitian ini, yang meliputi model YOLOv8n, YOLOv9t, YOLOv10n, dan YOLO26n.

Pada saat *pipeline* dijalankan untuk pertama kalinya, proses konversi model dilakukan secara otomatis, di mana format model awal (.onnx) dikompilasi menjadi *engine* TensorRT berpresisi FP16 yang spesifik untuk arsitektur GPU target. Perlu dicatat secara khusus bahwa model YOLOv10n dan YOLO26n secara arsitektural telah didesain dengan sifat "*NMS-free*". Hal ini berarti bahwa kedua model tersebut tidak lagi membutuhkan tahapan *Non-Maximum Suppression* (NMS) yang terpisah pada *post-processing*, karena arsitektur jaringan telah dilatih secara inheren untuk menghasilkan prediksi akhir tanpa duplikasi *bounding box*.

2.5.3 Implementasi Pipeline Baseline

Arsitektur aliran data atau *pipeline baseline* yang dirancang dibangun berdasarkan elemen-elemen standar GStreamer dalam kerangka DeepStream. Aliran proses direpresentasikan mulai dari elemen masukan video (kamera ZED atau fail rekaman), yang kemudian diteruskan ke elemen nvstreammux untuk tahap *batching*. Aliran data selanjutnya diproses oleh nvinfer sebagai *Primary GPU Inference Engine* (GIE) untuk melakukan inferensi deteksi, dilanjutkan ke elemen nvtracker (NvDCF) untuk pelacakan objek, nvvideoconvert untuk konversi format, dan nvsosd untuk penggambaran *bounding box* serta label pada *frame*. Aliran diakhiri pada elemen *output sink* untuk menampilkan atau menyimpan hasil.

Penekanan utama pada implementasi arsitektur ini adalah pemanfaatan *buffer* NVMM (*NVIDIA Memory Manager*) yang memfasilitasi operasi *zero-copy*. Melalui integrasi *buffer* tersebut, keseluruhan tahapan pemrosesan mulai dari decode hingga *tracking* dieksekusi dan dijaga sepenuhnya di dalam memori GPU, sehingga perpindahan data secara bolak-balik antara CPU dan GPU yang memakan waktu komputasi besar dapat dihindari.

[Gambar 3: Diagram aliran data arsitektur pipeline baseline NVIDIA DeepStream beserta elemen NVMM]

2.5.4 Implementasi Optimasi Efficient NMS

Untuk meminimalisasi latensi pada tahap pasca-pemrosesan, implementasi optimasi diterapkan menggunakan *plugin* bawaan TensorRT, yakni EfficientNMS_TRT. Penerapan *plugin* ini secara spesifik dikhususkan pada arsitektur YOLOv8n dan YOLOv9t yang masih mensyaratkan tahap penyaringan kotak pembatas secara manual. Cara kerja dari optimasi ini adalah dengan memindahkan eksekusi algoritma *Non-Maximum Suppression* (NMS) yang biasanya beroperasi secara sekuensial dan membebani komputasi CPU. Eksekusi tersebut dialihkan dan diubah menjadi operasi paralel penuh di dalam kernel GPU sebagai bagian turunan (*tail node*) dari *graph* TensorRT. Dengan pendekatan ini, latensi *post-processing* dapat direduksi karena tahap NMS diintegrasikan langsung ke dalam *engine* inferensi.

[Gambar 4: Skema cara kerja NMS sekuensial di CPU dibandingkan dengan plugin EfficientNMS_TRT paralel di GPU]

2.5.5 Implementasi Multi Object Tracking

Tahap pelacakan objek (*object tracking*) diimplementasikan dengan mengintegrasikan elemen nvtracker yang diletakkan tepat setelah proses deteksi oleh nvinfer. Implementasi ini difokuskan pada pengonfigurasiannya dan perbandingan efisiensi dua algoritma pelacakan utama dari DeepStream, yaitu NvDCF dan NvSORT.

Algoritma NvDCF menggunakan pendekatan berbasis fitur (*feature-based*) yang melibatkan pemrosesan data piksel secara penuh untuk mengekstraksi korelasi visual objek, sehingga memiliki tingkat akurasi yang tinggi namun mensyaratkan komputasi yang berat. Di sisi lain, NvSORT dievaluasi menggunakan pendekatan murni berbasis pergerakan (*motion-only*) yang diturunkan dari *Kalman filter* dan algoritma asosiasi Hungarian. NvSORT dapat dieksekusi dengan sangat ringan secara komputasi

dikarenakan algoritma ini beroperasi tanpa melakukan pemrosesan piksel sama sekali. Penekanan ditegaskan bahwa implementasi pelacakan ini difokuskan murni untuk mengukur metrik efisiensi komputasi, seperti penggunaan sumber daya (*resource*) dan latensi pemrosesan pada perangkat *edge*. Sistem evaluasi pelacakan ini tidak dirancang untuk mengukur metrik kualitas ketepatan pelacakan itu sendiri, seperti jumlah *ID Switch* maupun perhitungan *Multiple Object Tracking Accuracy* (MOTA).

[Gambar 5: Komparasi karakteristik pemrosesan algoritma NvDCF dan NvSORT pada modul nvtracker]

2.5.5 Pengujian

Sistem evaluasi dan *benchmarking* dirancang melalui pengembangan aplikasi kustom berbasis bahasa C++ yang mengadopsi mekanisme *pad probe* dari antarmuka pemrograman aplikasi (API) GStreamer. Agar tidak menyebabkan blokir (*blocking*) atau mengganggu jalur rendering kritis pada *pipeline* utama, mekanisme *logging* atau pencatatan metrik per *frame* ke dalam berkas csv dieksekusi secara asinkron. Proses asinkron ini dijalankan pada *thread* pencatatan yang terpisah menggunakan antrean GAsyncQueue.

Berbagai metrik komputasi yang diukur dalam perancangan evaluasi ini meliputi *throughput* dengan target laju waktu nyata lebih dari atau sama dengan 30 FPS, serta perhitungan latensi pada masing-masing komponen perangkat lunak (seperti Lat_PreMux, Lat_Infer, Lat_Tracker, dan lain-lain). Pengukuran juga dieksekusi secara konkuren untuk merekam status utilitas perangkat keras, yang meliputi persentase penggunaan unit GPU, penggunaan CPU, pemakaian kapasitas RAM, serta tingkat konsumsi daya sistem menggunakan alat ukur tegrastats.

[Gambar 6: Arsitektur mekanisme asynchronous logging menggunakan GAsyncQueue dan pad probe pada pipeline]

BAB III HASIL DAN PEMBAHASAN

Bab ini menguraikan secara mendalam hasil pengujian dan analisis terhadap kinerja *real-time pipeline* berbasis NVIDIA DeepStream pada perangkat Jetson Orin Nano. Pengujian ini didasarkan pada eksekusi 60 *run* secara berurutan, yang merupakan kombinasi dari 6 varian konfigurasi model deteksi (termasuk optimasi NMS) dan 2 konfigurasi algoritma *tracking* (NvDCF dan NvSORT), dengan masing-masing konfigurasi diulang sebanyak 5 kali (*repetisi*). Evaluasi difokuskan pada pemenuhan kriteria metrik waktu nyata (*throughput* dan *latensi*), utilitas sumber daya perangkat keras, pembuktian stabilitas deteksi secara *as-deployed*, serta sintesis *trade-off* antara kecepatan dan efisiensi komputasi guna menjawab ketiga rumusan masalah penelitian.

3.1 Kondisi Eksekusi dan Parameter Baseline

Untuk menjamin validitas dan asas keterulangan (*reproducibility*) eksperimen dalam pengujian instrumen perangkat keras, kondisi lingkungan dieksekusi secara ketat. Seluruh *run* dijalankan menggunakan berkas masukan video yang identic untuk menghilangkan variabel bebas dari densitas lalu lintas yang fluktuatif. Mode daya (*power mode*) pada perangkat dikunci pada indeks mode 0 (10W), yang merupakan mode performa tertinggi yang tersedia secara spesifik pada varian System-on-Chip (SoC) Jetson Orin Nano 4GB yang digunakan.

Selain itu, utilitas `jetson_clocks` diaktifkan untuk mengunci frekuensi CPU dan GPU pada batas maksimalnya, guna meminimalisasi *jitter* yang disebabkan oleh penskalaan frekuensi dinamis oleh sistem operasi. Secara prosedural, jeda pendinginan (*cooldown*) selama 60 detik serta pembersihan *cache* memori diterapkan di antara setiap skenario untuk menstabilkan kondisi termal dan memitigasi anomali akibat *thermal throttling*.

Parameter	Nilai Konfigurasi	Keterangan
Mode Daya	10W	Konfigurasi daya rendah p
Status <code>jetson_clocks</code>	Aktif	
Repetisi Skenario	5 Kali	
Jeda Termal	60 Detik	
Presisi Runtime	TensorRT FP16	

[Tabel 3.1: Parameter Konfigurasi Pengujian Perangkat Keras]

(Catatan Visual: Tabel ini harus memuat tiga kolom: "Parameter", "Nilai Konfigurasi", dan "Keterangan". Baris di dalamnya mencakup: Mode Daya (10W - Indeks 0), Status `jetson_clocks` (Aktif), Repetisi Skenario (5 kali), Jeda Termal (60 detik), Interval Logging `tegrastats` (1000 ms), dan Presisi Runtime (TensorRT FP16). Tabel ini krusial untuk membuktikan validitas eksperimen hardware Anda).

3.2 Hasil Pengujian Kinerja Baseline Pipeline

Sub-bab ini menguraikan performa mentah sistem sebelum integrasi optimasi parsial diterapkan, di mana keempat arsitektur model (YOLOv8n, YOLOv9t, YOLOv10n, dan YOLOv26n) dijalankan bersama *tracker* bawaan (*default*) yaitu NvDCF.

3.2.1 Analisis Throughput (FPS)

Throughput merupakan indikator fundamental bagi keandalan sistem deteksi pada kendaraan cerdas (ADAS). Dalam penelitian ini, ambang batas *real-time* ditetapkan secara konservatif pada angka 30 *Frames Per Second* (FPS). Berdasarkan hasil pengukuran, keempat arsitektur model secara konsisten berhasil melampaui ambang batas tersebut dengan margin yang sangat memadai.

Model	Average FPS	Median FPS	Standar Deviation	Ambang Batas Real-Time
YOLOv8n	66.77	66.52	0.51	123%
YOLOv9t	51.52	51.73	0.61	72%
YOLOv10n	67.02	67.22	0.40	123%
YOLOv26n	65.66	65.45	1.02	119%

[Tabel 3.2: Kinerja Throughput Model Baseline]

(Catatan Visual: Tabel berisikan kolom "Nama Model", "Avg FPS", "Median FPS", "Std Deviasi", "Min FPS", "Max FPS", dan "Status Kelulusan vs 30 FPS". Nilai yang dimasukkan: YOLOv10n (67,02 Avg FPS), YOLOv8n (66,77 Avg FPS), YOLOv26n (65,66 Avg FPS), dan YOLOv9t (51,52 Avg FPS). Berikan highlight atau warna khusus pada kolom status kelulusan).

YOLOv10n mencatatkan *throughput* tertinggi dengan rata-rata 67,02 FPS, sementara YOLOv9t tercatat sebagai model dengan laju terendah pada angka 51,52 FPS. Fakta krusial dari pengujian ini adalah kestabilan metrik; seluruh 60 *run* individual membuktikan bahwa titik terendah (nilai minimum FPS) pada model paling lambat sekalipun masih berada di angka 50,88 FPS ($\pm 70\%$ di atas ambang batas 30 FPS). Hal ini mengonfirmasi bahwa eksekusi *pipeline inference* pada Jetson Orin Nano, secara prinsip, sepenuhnya memadai untuk skenario operasi *real-time*.

[Grafik Boxplot 3.1: Distribusi FPS Baseline Berdasarkan Model]

(Catatan Visual: Grafik Boxplot dengan sumbu horizontal mewakili 4 model baseline, dan sumbu vertikal mewakili nilai FPS. Grafik ini berfungsi untuk memvisualisasikan "Std Deviasi" yang sangat kecil, menunjukkan bahwa dari 5 kali repetisi per model, performa Jetson sangat stabil dan minim pencilan/outlier negatif).

3.2.2 Analisis Latensi Keseluruhan (End-to-End)

Untuk sistem berbasis *safety-critical*, kelayakan tidak hanya diukur melalui nilai rata-rata, melainkan juga melalui sebaran keterlambatan pemrosesan atau *jitter*. Oleh karena itu,

latensi dalam sistem ini diukur dengan memperhitungkan Persentil 95 (p95) dan Persentil 99 (p99).

Model	Average Latensi (ms)	Average P95 Latensi (ms)
YOLOv8n	273.35	329.10
YOLOv9t	457.67	536.32
YOLOv10n	247.48	299.46
YOLOv26n	355.56	409.20

[Tabel 3.3: Distribusi Latensi End-to-End]

(Catatan Visual: Tabel yang membandingkan Latensi Median, Latensi p95, dan Latensi p99 dalam satuan milidetik (ms) untuk keempat model).

Hasil analisis memperlihatkan bahwa YOLOv9t bertindak sebagai anomali (*outlier*) struktural. Model ini memiliki median latensi 454,38 ms, namun melonjak pada persentil 99 hingga mencapai 547,15 ms. Rentang *jitter* sebesar ~93 ms pada YOLOv9t jauh melebihi variabilitas model lain (misalnya YOLOv8n yang hanya bergeser ~46 ms). Karakteristik sebaran *jitter* yang lebar ini mengindikasikan bahwa YOLOv9t memiliki determinisme latensi yang buruk, yang berpotensi mencederai tenggat waktu respons peringatan dini (*early warning response*) pada implementasi ADAS tingkat lanjut.

3.2.3 Dekomposisi Latensi Per-Komponen Pipeline

Guna mengidentifikasi titik leher botol (*bottleneck*) secara lebih presisi, waktu tunggu diurai menjadi tujuh segmen *probe* pada GStreamer (Mux, Infer, Tracker, dll).

[Grafik Stacked Bar 3.2: Rincian Latensi Per-Komponen Pipeline]

(Catatan Visual: Grafik batang bertumpuk (Stacked Bar Chart). Sumbu X adalah keempat model baseline. Sumbu Y adalah waktu latensi (ms). Setiap batang diwarnai secara proporsional berdasarkan *Lat_PreMux*, *Lat_Mux*, *Lat_Infer*, *Lat_Tracker*, *Lat_PreOSD*, *Lat_OSD*, dan *Lat_Output*. Bagian batang *Lat_PreMux* pada YOLOv9t harus divisualisasikan dominan panjang untuk menunjukkan *bottleneck*).

Berdasarkan analisis per-komponen, membengkaknya latensi keseluruhan pada YOLOv9t ternyata bukan berpusat pada komponen inferensi (*Lat_Infer_ms*), melainkan pada antrean di depan *multiplexer* (*Lat_PreMux_ms*) yang menembus angka 360,92 ms. Tingginya nilai ini dianalisis sebagai gejala *backpressure*; arsitektur jaringan sekuensial (PGI/GELAN) pada YOLOv9 menyebabkan beban tumpuk saat bertemu dengan *tracker* yang kompleks (NvDCF) di hilir *pipeline*. Di sisi lain, pembuktian empiris juga memvalidasi bahwa GFLOPs teoretis tidak berbanding lurus dengan performa perangkat *edge*. YOLO26n yang memiliki GFLOPs terendah (5,2) ternyata tidak menjadi model tercepat (hanya 65,66 FPS), akibat struktur memori komputasi yang merambat ke komponen *tracker*.

3.3 Hasil Pengujian Optimasi NMS Paralel (Menjawab Rumusan Masalah

2)

Penelitian ini mencoba menyisipkan *plugin* EfficientNMS_TRT dari TensorRT untuk menggantikan *Non-Maximum Suppression* (NMS) berbasis iterasi CPU menjadi berbasis paralelisasi GPU. Pengujian ini diterapkan pada model YOLOv8n dan YOLOv9t.

3.3.1 Dampak EfficientNMS terhadap Latensi Inferensi

Penerapan *plugin* ini membuahkan hasil positif secara mikro pada tahap inferensi. Lat_Infer_ms pada YOLOv8n menurun sebesar 0,24 ms (dari 19,21 ms menjadi 18,97 ms) dan pada YOLOv9t menurun sebesar 0,56 ms. Walaupun konsisten menurun, selisih waktu dalam hitungan sub-milidetik ini membuktikan bahwa NMS bawaan pada skala model kelas *nano/tiny* pada dasarnya memang bukan *bottleneck* komputasi yang signifikan.

Model	Efficient NMS	Average Latensi Inference (ms)
YOLOv10n	Tidak	16.72
YOLOv26n	Tidak	22.88
YOLOv8n	Tidak	19.21
	Ya	18.97
YOLOv9t	Tidak	28.65
	Ya	28.09

[Tabel 3.4: Perbandingan Lat_Infer_ms Baseline vs. EfficientNMS]

(Catatan Visual: Tabel sederhana yang membandingkan Lat_Infer_ms pada kondisi baseline dan kondisi optimasi NMS paralel untuk membuktikan penurunan mikro tersebut).

3.3.2 Dampak EfficientNMS terhadap Throughput Keseluruhan

Dampak komputasi mikro pada tahap sebelumnya secara kuantitatif ternyata gagal diterjemahkan menjadi peningkatan *throughput* secara makro.

⇒ Kerjakan nanti di jetson

[Tabel 3.5: Signifikansi Statistik Throughput Optimasi NMS]

(Catatan Visual: Tabel komparasi FPS dengan menyertakan perhitungan " Δ FPS" dan nilai kemaknaan statistik (*P-Value*) dari hasil uji Welch's *t-test*).

Uji signifikansi (Welch's *t-test*) membuktikan bahwa pada YOLOv8n, penurunan FPS akibat *plugin* ini bersifat tidak signifikan (*p-value* = 0,592). Sebaliknya, pada YOLOv9t,

integrasi EfficientNMS justru terbukti menurunkan *throughput* rata-rata sebesar 2,1% secara signifikan ($p\text{-value} = 0,023$). Temuan anomali negatif yang sah secara keilmuan ini disebabkan karena sifat arsitektural *plugin* tersebut yang berjalan secara sekuensial (*tail dependency*) setelah detektor, sehingga merusak kesempatan *pipelining* GPU antar-*frame*. Fenomena ini dikonfirmasi oleh penurunan drastis rerata utilisasi GPU pada YOLOv9t dari 87,7% menjadi 69,3% (indikasi besarnya kondisi *idle/bubble* pada GPU).

3.3.3 Analisis Komparatif Model NMS-free

Model YOLOv10n dan YOLO26n tidak diikutsertakan dalam sub-bab sebelumnya karena secara arsitektural dibangun secara *NMS-free*. Namun, hasil baseline menunjukkan bahwa arsitektur bebas NMS sekalipun (seperti YOLO26n) tidak menjamin perolehan *throughput* tertinggi. Hal ini mempertegas kesimpulan Rumusan Masalah 2: baik pendekatan akselerasi *plugin* NMS paralel maupun arsitektur *NMS-free*, dampaknya tereliminasi apabila titik kemacetan utama (*Lat_PreMux_ms* dan *Lat_Tracker_ms*) tidak diatasi.

3.4 Hasil Pengujian Efisiensi Komputasi Tracking

Eksperimen ini mengevaluasi pertukaran (*trade-off*) dari dua profil *tracking*: NvDCF yang berbasis pendekatan visual (mengekstrak fitur piksel) berbanding dengan NvSORT yang sepenuhnya beroperasi pada prediksi gerak tanpa analisis piksel (*motion-only Kalman/Hungarian*).

3.4.1 Perbandingan Latensi Murni Tracker

Dari sisi efisiensi absolut pada lapisan komponen, algoritma NvSORT membuktikan superioritas komputasinya di seluruh enam variasi *pipeline*.

Model	Median NvDCF (ms)	Median NvSORT (ms)	Rasio Percepatan
YOLOv8n	10.981	0.363	30x
YOLOv9t	18.935	0.723	26x
YOLOv10n	3.530	0.297	12x
YOLOv26n	13.633	0.428	32x

[Tabel 3.6: Komparasi *Lat_Tracker_ms* NvDCF vs NvSORT]

(Catatan Visual: Tabel yang membandingkan nilai Median NvDCF, Median NvSORT, dan Rasio percepatannya. Tampilkan dengan jelas rasio 10x hingga 62x yang didapat. Misal: YOLOv8n NvDCF 9,77ms vs NvSORT 0,29ms, rasio ~34x).

Waktu proses (*Lat_Tracker_ms*) menggunakan NvSORT stabil mendekati nilai nol (median < 0,4 ms), yang berarti secara komputasi 10× hingga 62× lebih cepat ketimbang proses penapisan korelasi fitur pada NvDCF. Temuan ini menegaskan bahwa properti

keringanan beban pada NvSORT bersifat universal tanpa dipengaruhi oleh jenis detektor objek di hulunya.

3.4.2 Dampak Algoritma Tracking terhadap FPS Keseluruhan

Dampak rasio ekstrem dari *tracker* terhadap *throughput* (FPS) secara keseluruhan justru tidak linier (*kondisional*).

[Grafik Boxplot Berdampingan 3.3: Distribusi FPS Berdasarkan Tracker]

(Catatan Visual: Kelompokkan boxplot berdasarkan nama model. Di dalam tiap nama model, sandingkan dua boxplot: kotak warna biru untuk NvDCF, dan warna oranye untuk NvSORT. Grafik ini secara visual akan menonjolkan perbedaan drastis pada YOLOv9t, di mana kotak NvSORT akan meloncat 30% ke atas, sementara untuk model lain kotak biru dan oranye tampak sejajar).

Pada arsitektur ringan (YOLOv8n, v10n, v26n), lonjakan FPS nyaris tak terlihat (hanya berkisar +0,09 hingga +1,63 FPS) karena beban laten dari NvDCF sukses terkamuflase oleh ruang sisa waktu antar-frame (*slack headroom*). Akan tetapi, pada *pipeline* YOLOv9t yang padat beban, peralihan dari NvDCF menuju NvSORT menghasilkan peningkatan komputasi masif, melesatkan kecepatan inferensi secara ekstrem dari kisaran ~50 FPS menuju stabil di ~67 FPS (peningkatan +30% FPS, *p-value* < 0.0001). Ini membuktikan bahwa rekomendasi generik spesifikasi teknis tidak dapat berdiri sendiri; *tracker* ringan baru membuahkan peningkatan sistem secara kasatmata apabila *headroom* sistem sudah berstatus kritis.

3.4.3 Analisis Efisiensi Sumber Daya Perangkat Keras

Guna merangkum profil komputasi seutuhnya, pembacaan register energi dari tegrastats diagregasikan.

Model	Efficient NMS	Tracker	GPU (%)	RAM (MB)	Daya VDD_IN (mW)	Energi/Frame (mJ)
YOLOv10n	-	NvDCF	66,9	1449,1	8614,0	128,5
	-	NvSORT	65,6	1388,4	8569,2	127,3
YOLOv26n	-	NvDCF	75,6	1418,0	8827,6	134,5
	-	NvSORT	68,1	1389,5	8455,5	125,7
YOLOv8n	-	NvDCF	84,6	1368,0	9185,8	137,6
	-	NvSORT	64,3	1296,5	8447,3	126,3
	Ya	NvDCF	87,2	1371,3	9202,0	138,1
	Ya	NvSORT	57,4	1310,7	8443,2	125,6
YOLOv9t	-	NvDCF	87,7	1365,2	8598,3	166,9
	-	NvSORT	90,6	1300,5	8930,2	133,1

	Ya	NvDCF	69,3	1361,7	8389,2	166,3
	Ya	NvSORT	91,2	1318,2	8942,3	133,1

[Tabel 3.7: Metrik Utilisasi Perangkat Keras dan Energi]

(Catatan Visual: Tabel berisi metrik GPU (%), RAM (MB), Tegangan Daya VDD_IN (mW), dan kolom kalkulasi sekunder: Estimasi Energi/Frame (mJ) yang didapatkan dari pembagian Daya dengan rerata FPS).

Analisis utilisasi memperlihatkan bahwa NvSORT secara konsisten menekan nilai energi yang dikeluarkan per-satu unit pekerjaan (*Energi/frame* dalam millijoule). Menariknya, pada YOLOv9t menggunakan NvSORT, utilisasi GPU melonjak ke ~90,6% (vs 87,7% NvDCF), sedangkan konsumsi daya sesaatnya (VDD_IN) membesar (8.930 mW). Ini adalah manifestasi dari terbukanya sumbatan *bottleneck*; hilangnya beban berat NvDCF memicu GPU mengeksekusi kerangka deteksi YOLOv9t jauh lebih agresif (*GPU-bound*), yang bermuara pada ledakan *throughput*. Adapun batasan peranti *edge* terbukti pada realitas utilisasi RAM, di mana satu arus video ADAS sudah memonopoli 1.265 hingga 1.449 MB, atau ~35% dari plafon 4GB yang dimiliki Orin Nano.

3.5 Verifikasi Akurasi As-Deployed FP16 (Uji Sanity Check)

Bagian ini dikonstruksikan sebagai uji integritas arsitektural (*sanity check*), untuk memverifikasi bahwasanya determinasi kecepatan komputasi ekstrem (menggunakan optimasi model FP16) tidak merusak logika deteksi (*reasoning detection*) di level praktikal.

3.5.1 Evaluasi Akurasi Baseline (Proxy FP32)

Secara *proxy*, pengujian akurasi penuh (FP32) di *cloud GPU* (Tesla T4) membuktikan daya saing model. YOLOv8n memimpin perolehan presisi maksimum dengan parameter mAP50-95 sebesar 0,8397. Namun demikian, YOLOv10n juga memperlihatkan kapabilitas yang tak kalah presisi (mAP50-95: 0,8370) sembari menggunakan 20% ruang teoretis komputasi (GFLOPs) yang lebih sedikit dari versi YOLOv8n.

[Tabel 3.8: Metrik Akurasi Standar Proxy FP32]

(Catatan Visual: Susun berjejer perbandingan Params, GFLOPs, mAP50, mAP50-95, Precision, dan Recall antara keempat model).

3.5.2 Analisis Deviasi Akurasi Kuantisasi FP16

Tahapan dekompresi bit dari FP32 ke format representasi *half-precision* (FP16 TensorRT) perlu dievaluasi deviasi geometrisnya. Melalui pemetaan *bounding box* langsung dari keluaran metadata NvDsObjectMeta pada jalur pipa berjalan (*as-deployed pipeline*), penurunan kualitas (*degradation*) diukur terhadap referensi label *ground truth* dari set validasi KITTI. Pembuktian tingkat degradasi sempit (marjinal) mendasari validitas optimasi perangkat keras yang didiskusikan pada tiga sub-bab sebelumnya,

sekaligus memberikan jaminan integritas pada eksekusi persepsi keamanan tingkat satu di sistem ADAS.

[Tabel 3.9: Deviasi Akurasi Kuantisasi FP32 vs FP16 TensorRT]

(Catatan Visual: Tabel ini disiapkan khusus untuk menampung selisih metrik/Delta (Δ mAP50, Δ mAP50-95). Jika deviasi sangat sempit (misal $< 0.5\%$), ini menjadi pembenaran kuat untuk keputusan optimasi Anda).

[Ilustrasi 3.1: Perbandingan Visual Deteksi Bounding Box As-Deployed]

(Catatan Visual: Susun tiga buah gambar hasil tangkap layar (screenshot) berdampingan dalam satu baris. Gambar (1) Ground Truth Label dari Dataset KITTI. Gambar (2) Hasil Prediksi Model FP32 (Cloud). Gambar (3) Hasil Prediksi Pipeline Deepstream FP16 (Edge). Ilustrasi visual ini sangat penting dalam penulisan ilmiah computer vision untuk mendemonstrasikan bahwa tidak ada perbedaan drastis yang merusak algoritma pembacaan objek).

3.6 Pembahasan Akhir dan Analisis Trade-Off

Bagian akhir mensintesis serangkaian variabel untuk mengekstrak rekomendasi empiris aplikatif untuk adopsi industri.

3.6.1 Kompromi Kecepatan, Akurasi, dan Efisiensi Energi

Pemilihan sistem ideal (*Pareto Frontier*) pada ADAS Edge AI tidak bertumpu pada satu nilai metrik absolut tunggal, namun dikondisikan pada kebutuhan rancang bangun rekayasa (*engineering constraints*):

1. **Kondisi Akurasi Superior:** Apabila prioritas ada pada akurasi prediksi mutlak (mAP_{50-95} puncak), integrasi **YOLOv8n yang dikawinkan dengan NvSORT** direkomendasikan. Walaupun latensinya memakan memori sedikit lebih besar, efisiensi NvSORT mampu menekan besaran energinya ke level minimal.
2. **Kondisi Efisiensi Ekstrem:** Apabila perangkat difokuskan menekan *GFLOPs*, maka kombo **YOLO26n bersama NvSORT** merupakan jawaban.
3. **Keseimbangan Optimal (Titik Pareto-Frontier):** **YOLOv10n** merupakan kandidat *default* terkuat. Arsitekturnya mampu bertoleransi terhadap penggunaan tipe *tracker* apa saja, sekaligus menyajikan kecepatan tertinggi (67 FPS) sembari menjaga margin mAP nyaris setara dengan batas maksimal.
4. Sebaliknya, **YOLOv9t** terbukti tidak kompatibel secara operasional apabila dijalankan menggunakan konfigurasi standar (NvDCF), lantaran risiko *backpressure* latensi *PreMux*-nya mengancam keamanan batas respons ADAS waktu nyata.

[Grafik Scatter Plot 3.4: Peta Pareto Trade-off Kinerja ADAS]

(Catatan Visual: Buat grafik Scatter Plot (diagram pencar). Sumbu X mendefinisikan Energi per-Frame (mJ) atau kecepatan FPS. Sumbu Y adalah tingkat Akurasi (mAP50-95). Titik-titik plot mewakili kombinasi (Contoh: "v8n+SORT", "v9t+DCF"). Tarik satu buah garis kurva eksponensial (Pareto Curve) yang menyentuh titik-titik terluar atas. Titik yang jatuh menjauhi garis lengkung tersebut (seperti v9t+DCF) terbukti gagal menyeimbangkan trade-off performa secara kasat mata).

3.6.2 Analisis Keterbatasan Sistem Terpasang (Edge Device)

Rangkaian eksperimen ini tidak luput dari batasan fisik unit. Tingginya penetrasi memori (hingga ~35% dari RAM) mengunci Jetson Orin Nano 4GB dalam limitasi topologi "*satu kamera, satu model*". Setiap upaya ekspansi fungsional (misal; arsitektur sistem ADAS berbasis multi-kamera keliling 360 derajat) akan menghadapi jurang kehabisan memori seketika.

Metodologi penanggulangan batas suhu (*thermal throttling*) melalui siklus *cooldown* antar-skenario 60 detik terbukti menyelamatkan konsistensi pengujian. Namun demikian, skenario nyata kendaraan yang bermobilisasi pada intensitas sinar matahari (*outdoor temperature*) akan menuntut manajemen sistem pendinginan yang independen yang berada di luar jangkauan optimasi arsitektural perangkat lunak pada penelitian ini. Terakhir, seluruh deteksi model saat ini masih beroperasi pada bidang planar (2D). Di waktu mendatang, pengolahan citra stereoskopis ganda langsung menuju estimasi dimensi titik awan (*point-cloud* 3D) diperlukan guna mengalkulasi proksimitas akurat bahaya ruang kemudi secara konkret.

BAB IV KESIMPULAN

4.1 Kesimpulan

Berdasarkan hasil pengujian dan pembahasan terhadap 60 *run* eksekusi (kombinasi 6 model, 2 konfigurasi *tracker*, dan 5 repetisi) pada *pipeline* Nvidia DeepStream menggunakan perangkat *edge* NVIDIA Jetson Orin Nano, penelitian ini menarik kesimpulan sebagai berikut:

1. **Kinerja *Real-Time Pipeline* Nvidia DeepStream (Menjawab Rumusan Masalah 1)** Keempat model deteksi *baseline* (YOLOv8n, YOLOv9t, YOLOv10n, dan YOLO26n) terbukti secara konsisten berhasil berjalan secara *real-time* dan melampaui ambang batas 30 FPS dengan margin yang besar. Throughput rata-rata berada pada rentang 51,52 FPS (YOLOv9t) hingga 67,02 FPS (YOLOv10n). Dari sisi latensi per-komponen, YOLOv9t teridentifikasi sebagai *outlier* dengan latensi *end-to-end* tertinggi (median 454,38 ms, p99 547,15 ms). Tingginya latensi pada YOLOv9t utamanya disebabkan oleh *backpressure* pada tahap Lat_PreMux_ms (360,92 ms) akibat interaksi arsitektur yang lebih sekuensial dengan beban komputasi *tracker* di tahap hilir. Selain itu, hasil pengujian menegaskan bahwa spesifikasi GFLOPs teoretis tidak selalu berbanding lurus dengan performa aktual di perangkat *edge*; contohnya, YOLO26n yang memiliki GFLOPs terendah tidak menjadi model dengan *throughput* tercepat.
2. **Evaluasi Optimasi *Non-Maximum Suppression* (NMS) Paralel (Menjawab Rumusan Masalah 2)** Penerapan *plugin* TensorRT EfficientNMS tidak terbukti memberikan peningkatan efisiensi *pipeline* yang signifikan pada model YOLOv8n maupun YOLOv9t. Walaupun metrik Lat_Infer_ms membaik, perubahannya hanya berada pada skala sub-milidetik. Pada tingkat *throughput* keseluruhan, YOLOv8n tidak menunjukkan selisih FPS yang signifikan secara statistik, sedangkan performa YOLOv9t justru mengalami penurunan sebesar 2,1% secara signifikan. Temuan ini membuktikan bahwa tahap inferensi bukanlah *bottleneck* utama pada kelas model *nano/tiny* di platform ini, melainkan tahap Lat_PreMux_ms dan Lat_Tracker_ms. Keberadaan *model NMS-free* (YOLOv10n dan YOLO26n) turut memvalidasi bahwa penghilangan NMS secara arsitektural tidak menjamin capaian FPS maksimal tanpa optimasi komponen *pipeline* lainnya.
3. **Efisiensi Komputasi Pemilihan Algoritma *Tracking* (Menjawab Rumusan Masalah 3)** Secara arsitektural level komponen, algoritma NvSORT terbukti jauh lebih efisien dengan biaya komputasi (Lat_Tracker_ms) 10× hingga 62× lebih rendah dibandingkan NvDCF secara konsisten pada seluruh model uji. Selain itu, NvSORT juga mengonsumsi energi per *frame* yang lebih rendah. Akan tetapi, implikasi penghematan ini terhadap *throughput* akhir bersifat kondisional. Peningkatan FPS yang masif (hingga +33%) hanya dirasakan ketika NvSORT dipasangkan dengan YOLOv9t, karena model tersebut telah kehabisan *headroom* waktu-per-*frame*. Sementara pada model lain (YOLOv8n, YOLOv10n, YOLO26n), pergantian ke NvSORT tidak memberikan selisih FPS yang berarti karena beban NvDCF masih mampu diserap oleh kelonggaran *slack* komputasi

yang tersedia. Dengan demikian, optimasi algoritma *tracking* tidak dapat dinilai secara terisolasi, melainkan harus diukur secara *end-to-end* bersama model deteksinya.

Sebagai sintesis keseluruhan, tidak ada konfigurasi tunggal yang terbaik. YOLOv8n dengan NvSORT direkomendasikan untuk prioritas akurasi maksimal, YOLOv26n dengan NvSORT untuk keseimbangan efisiensi komputasi, dan YOLOv10n sangat toleran terhadap pilihan *tracker* apapun (*Pareto-front*). Adapun rekomendasi terkait akurasi ini masih bersifat indikatif berdasar proksi *baseline* FP32, karena verifikasi mAP *as-deployed* presisi FP16 belum sepenuhnya dieksekusi.

4.2 Saran

Berdasarkan kesimpulan dan batasan masalah yang ditemukan selama penelitian, terdapat beberapa saran teknis dan metodologis untuk pengembangan sistem ADAS (*Advanced Driver Assistance Systems*) di masa mendatang:

1. **Penyelesaian Verifikasi Akurasi As-Deployed FP16:** Prioritas utama penelitian lanjutan adalah mengeksekusi uji mAP presisi FP16 secara langsung pada perangkat keras Jetson Orin Nano. Hal ini krusial untuk memastikan bahwa klaim kompromi akurasi (*trade-off*) yang digunakan murni bersumber dari operasional sesungguhnya, bukan sekadar nilai proksi FP32 di komputasi *cloud*.
2. **Pengukuran Termal Terintegrasi:** Implementasi sistem pada kendaraan sesungguhnya rentan terhadap panas berlebih. Disarankan agar instrumentasi pengukuran (seperti *LogParser* dari utilitas *tegrastats*) dikembangkan untuk mencatat data suhu SoC (*System-on-Chip*) secara langsung guna memvalidasi efektivitas mitigasi *thermal throttling*, dan tidak hanya bergantung pada jeda waktu pendinginan (*cooldown*).
3. **Eksplorasi Strategi Optimasi Lanjutan:** Mengingat *plugin* EfficientNMS tidak memberikan dampak signifikan, optimasi sebaiknya dialihkan pada penyesuaian score-threshold, reduksi *output* kotak pembatas (misal max-output-boxes di bawah 100), atau pemanfaatan arsitektur *NMS-free* bawaan secara penuh.
4. **Peluang Kuantisasi INT8:** Disarankan untuk melakukan eksperimen lebih lanjut menggunakan presisi INT8 pada perangkat keras yang mendukung spesifikasi DLA (*Deep Learning Accelerator*), sehingga rasio perbandingan kecepatan dan akurasi terhadap FP16 dapat dipetakan secara lebih komprehensif.
5. **Perluasan Skenario Pengujian Lintas Kondisi:** Model perlu diuji menggunakan klip video dengan durasi yang lebih panjang serta variasi lingkungan ekstrem, seperti kepadatan lalu lintas tinggi, kondisi minim cahaya (malam hari), atau cuaca buruk, untuk mengevaluasi generalisasi model *edge device*.
6. **Penambahan Metrik Kualitatif Tracking:** Untuk memvalidasi efisiensi NvSORT terhadap NvDCF secara utuh, penelitian ke depan harus memasukkan analisis metrik ketahanan kualitas pelacakan seperti jumlah *ID switch*, MOTA, dan IDF1.
7. **Pengujian Skalabilitas Infrastruktur:** Batas maksimal memori RAM yang digunakan pada pengujian ini mencapai $\pm 35\%$ dari kapasitas 4GB untuk *single-stream*. Diperlukan pengujian untuk sistem ADAS dengan konfigurasi *multi-camera* dan *multi-model* secara paralel guna mengetahui ambang batas limitasi *bottleneck* memori pada modul tersebut.

8. **Pemanfaatan Kemampuan *Depth-Sensing*:** Kamera ZED yang berjenis *stereo* dapat dioptimalkan tidak hanya sebagai penangkap citra 2D, tetapi dilanjutkan untuk proses estimasi jarak spasial objek secara 3D demi meningkatkan fungsi peringatan kewaspadaan pengendara pada sistem ADAS yang sesungguhnya.

DAFTAR PUSTAKA

- Amrullah, A. K. E (2021). Deteksi Genangan Sampah pada Permukaan Air Kanal dengan Menggunakan Metode Deep Learning. *Skripsi S1 Universitas Hasanuddin*
- Anggraini, N., Tawakkal, I., Rachman, I., dan Matsumoto, T. (2024). Object Detection of Macroplastic Waste Using Unmanned Aerial Vehicles in Urban Canal. *Ecological Engineering & Environmental Technology* 2024, 25(8), 271–284. <https://doi.org/10.12912/27197050/189888>
- Anisah, S. N. (2026). Perbandingan Akurasi Dan Waktu Inferensi Metode YOLOv10 YOLOv11 YOLOv12 Untuk Deteksi Lubang Jalan. *Skripsi S1 Universitas Hasanuddin*
- Chiang, C.H. dan Juang, J.G. (2023). Application of UAVs and Image Processing for Riverbank Inspection. *MDPI Machine*, 11, 876. <https://doi.org/10.3390/machines11090876>
- Dani, W. O. A. P. W. (2025). Deteksi Penyakit Buah Kakao Secara Remote Menggunakan Visi Komputer Melalui Drone Dengan Metode Deep Learning. *Skripsi S1 Universitas Hasanuddin*
- Gustriyadi, E, Sihombing, V, Masrizal, dan Adi, P.D.P. (2023). A Real-Time Web Information System Based On A Global Positioning System For Monitoring Environmental Pollution. *Jurnal Riset Informatika*, 5,1. <https://doi.org/10.34288/jri.v5i1.204>
- Gutsa T., Trois, C. Vries, R., dan Mani, T. (2024). Wasted shores: Using drones to monitor the spatio-temporal evolution of debris accumulation hotspots on South Africa's Umgeni River. *Science of the Total Environment*, 955.
- Hidayatullah, P., Syakrani, N., Sholahuddin, M. R., Gelar, T., dan Tubagus, R. (2026). YOLOv8 to YOLO11 Performance Benchmark and Comprehensive Architectural Comparative Review. *JURNAL RESTI (Rekayasa Sistem dan Teknologi Informasi)*. <https://doi.org/10.29207/resti.v10i2.6598>
- Kasim, A. I. (2026). Pengembangan Aplikasi Mobile Klasifikasi Tanda Tangan Berbasis CNN. *Skripsi S1 Universitas Hasanuddin*.
- Khanam, R. dan Hussain, M. (2024). YOLOv11: An Overview of the Key Architectural Enhancements. *Arxiv*.
- Kota Makassar. (2011). *Peraturan Daerah Kota Makassar Nomor 4 Tahun 2011 tentang Pengelolaan Sampah*.
- Madenda, S. (2024). Komputer Vision, Kecerdasan Artifisial, dan Sistem Tertanam (1st ed.). *Penerbit Gunadarma*.
- Pan, S., Yoshida, K., Boney, A.S., dan Nishiyama, S. (2022). The Application of Drone-assisted Deep Learning Technology in Riverbank Garbage Detection. *Journal of Japan Society of Civil Engineers*, 78, 2.
- Pfeiffer, R. dkk. (2023). Use of UAVs and Deep Learning for Beach Litter Monitoring. *Electronics*, 12, 198. <https://doi.org/10.3390/electronics12010198>
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 779–788).
- Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. *arXiv preprint, arXiv:1804.02767*.
- Republik Indonesia. (2008). *Undang-Undang Nomor 18 Tahun 2008 tentang*

Pengelolaan Sampah.

- Republik Indonesia. (2011). *Peraturan Pemerintah Republik Indonesia Nomor 38 Tahun 2011 tentang Sungai*.
- Santoso, U. R. N. S. dan Gamar, F. (2025). Deteksi Sampah Botol Plastik di Perairan Menggunakan YOLO v4Tiny. *Jteksis : Jurnal Teknologi Dan Sistem Informasi Bisnis*, 91-98
<https://doi.org/10.47233/jteksis.v5i1.1744>
- Schneider, F. Kunz, A., Hu, C.S., Yen, N. Dan Lin, H.T. (2021). Rapid-Survey Methodology to Assess Litter Volumes along Large River Systems— A Case Study of the Tamsui River in Taiwan. *MDPI Sustainability*, 13, 8765. <https://doi.org/10.3390/su13168765>
- Tjandra, B., Made, S. N. N., dan Christopher H., N. S. (2023). Deteksi Sampah di Permukaan dan Dalam Perairan pada Objek Video dengan Metode Robust and Efficient Post-Processing dan Tubelet-Level Bounding Box Linking. *Arxiv*
- Sharma, A., Kumar, V., dan Longchamps, L (2024). Comparative performance of YOLOv8, YOLOv9, YOLOv10, YOLOv11 and Faster R-CNN models for detection of multiple weed species. *Elsevier : Smart Agricultural Technology*, 9.
<https://doi.org/10.1016/j.atech.2024.100648>
- Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437.
<https://doi.org/10.1016/j.ipm.2009.03.002>

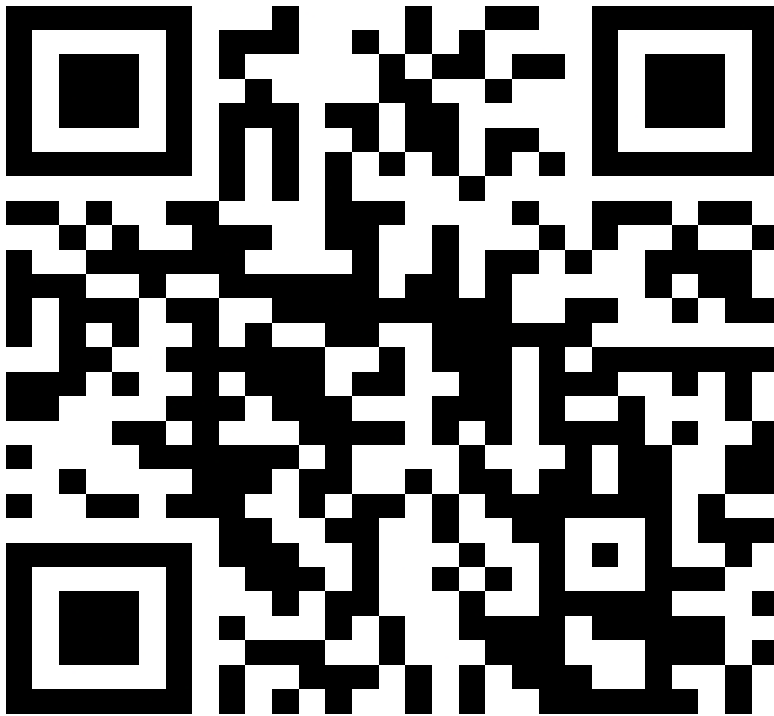
LAMPIRAN

Lampiran 1. Dataset



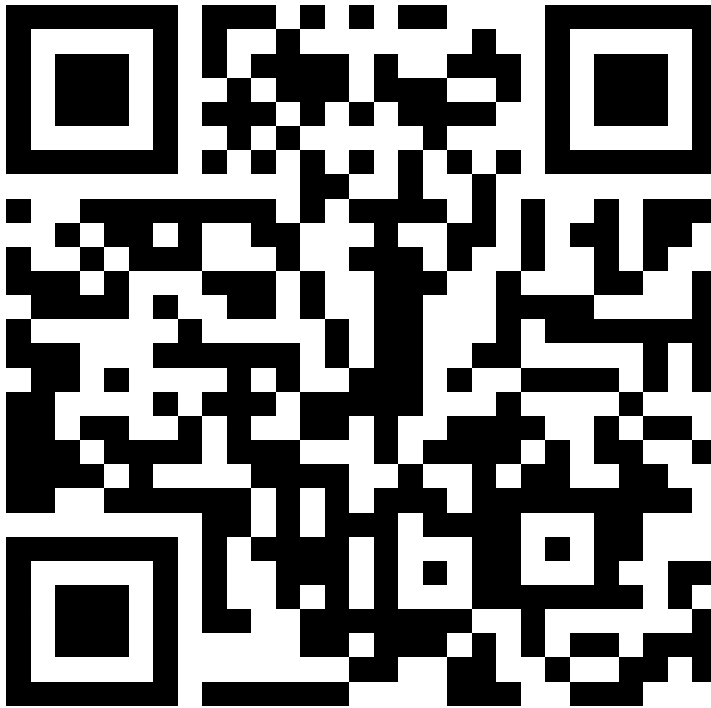
<https://universe.roboflow.com/trashdetection-p0qv7/bbox-sampah-revisi>

Lampiran 2. Source Code Website



<https://github.com/winati17/river-waste-detection>

Lampiran 3. Website Sistem



<https://river-waste-detection.vercel.app/>